

What Survives the Translation from Brain to Von Neumann Machine: Learning Is Pruning, and Four Mechanisms That Did Not Transfer

Dongin Kang
Independent Researcher
shaloooooooooom@gmail.com

Draft, October 2026

Abstract

Which brain mechanisms survive translation to a von Neumann machine? We ported five mechanisms and a follow-up to GPU training under one protocol: gradual synaptic pruning during learning, hippocampus-to-cortex consolidation in sleep, thalamic routing of what to process, predictive coding that skips the predictable, and spike-timing-dependent plasticity. One principle transfers. At matched final connection budgets, removing connections gradually while training beats training small, pruning once after training and dynamic sparse training (RigL at equal training length) at tight budgets, ties prune-after at loose ones, and its margin grows with sparsity: at 0.5% it reaches 97.8% on MNIST against 94.1%, 94.5% and 96.3%, and 89.0% on a CIFAR-10 ResNet-18 against 81.3%, 83.6% and 84.9%. The gain comes mostly from the gradual schedule, not from sparsity, and survives at matched inference FLOPs; pruning while learning adds up to a point at a quarter to a third of the cost of pruning after training and $3\text{--}59\times$ that of training small. A pre-trained start adds 3.5–5.3 points over pruning from scratch and saves no adaptation compute (93.7% on CIFAR-10 at 5% of an ImageNet-pre-trained ResNet-18’s connections); RigL from the inherited mask comes within 0.4 points at 28–42% of the compute, and at 0.5% the start breaks even only given time at the final density. The best local rule, synaptic drive, ends about a point behind per-layer magnitude pruning on the MLP and collapses on the CNN unless normalised per neuron. Four structures fail: sleep consolidation matches class-balanced replay; thalamic routing picks tokens no better than random and, as pre-routed attention, saves only 3.6–4.7% of compute; predictive coding neither picks tokens better than chance nor fills gaps better than leaving them; per-input subnetworks are input-specific yet worse than a fixed mask. STDP alone sparsifies a spiking network.

1 Introduction

A brain computes, stores and learns in the same synapses; a von Neumann machine keeps its processor and its memory apart and moves every weight between them. Most brain-inspired proposals for deep learning copy one brain mechanism into this second kind of machine and report whether it helps. We ask the question the other way round: when the same protocol and a common set of metrics are applied to each of several brain mechanisms, which of them survive the translation, and which do not?

Synaptic density in human cortex peaks in early childhood and falls by roughly half by adulthood [15]; the pruning is activity dependent and interleaved with learning. Deep learning takes the opposite route: parameters are accumulated from random initialisation, and sparsity, when wanted, is imposed afterwards [13, 14]. The lottery-ticket literature showed that large networks contain small trainable subnetworks [8], gradual magnitude pruning [30] and dynamic sparse training [7, 21]

showed that sparsity can be reached during training, and large-scale comparisons found plain magnitude pruning hard to beat [10]. That gradual removal beats one-shot removal is the premise of gradual magnitude pruning and of iterative lottery-ticket pruning, and that it stays ahead of RigL at equal training length was reported by Evci et al. [7]; we claim neither as new. What has not been laid out side by side is the developmental question at the budgets where the routes separate: at 0.5–2% of the connections, with training cost, inference FLOPs and sample efficiency accounted together, does it matter whether a network is pruned *while* it learns, after it learns, or never grown large in the first place, and does the answer survive a pre-trained start and local, activity-based criteria?

This paper is the first empirical instalment of a programme that ports brain mechanisms to ordinary GPU training under one protocol and asks which survive. Five mechanisms were ported, plus one follow-up suggested by the first result: gradual synaptic pruning during learning, a fast hippocampus consolidating into a slow cortex during sleep, thalamic routing that decides what to process, predictive coding that skips the predictable, spike-timing-dependent plasticity, and per-input pruning at inference time. Table 5 is the reference list: it maps each mechanism to its reduction and outcome, and the counts used elsewhere (one principle, four structures, the local rules) refer to its rows. We report the one principle that survived contact with the data, and the rest honestly as negative or mixed results, because they constrain what a brain-inspired architecture should and should not borrow; the hardware reading of the failures, compute–memory unity, appears only as a hypothesis in Section 6.

Contributions.

1. **A matched-budget comparison of four routes to sparsity** (Section 4). On MNIST (MLP, five budgets from 10% to 0.5% of a 1.86M-connection network) and CIFAR-10 (VGG-style CNN, three budgets from 10% to 1% of 2.2M connections), prune-during with a global magnitude criterion is the most accurate route at every budget of 2% and below, ties with prune-after (within 0.4 points) at the loosest budgets, and its margin over the additive baseline grows from 0.3 to 3.7 points (MNIST) and from 3.0 to 8.6 points (CIFAR-10) as the budget shrinks. A ResNet-18 replication (11.2M weights, three budgets from 5% to 0.5%) reproduces the winner and the growth of the margin, with prune-during at 0.5% reaching 89.0% against 81.3% (dense small), 83.6% (prune-after) and 84.9% (RigL at equal training length).
2. **The benefit is mostly in the gradual schedule; pruning while learning adds up to a point.** Pruning the same network once after training collapses accuracy at the pruning step (to below 60% at 0.5% density on MNIST) and recovers only partially; pruning it gradually after training, with the same schedule and the same total steps, recovers most of the loss (96.9% against 94.5% at 0.5% on MNIST; 84.8% against 81.1% at 1% on the CNN when fine-tuning runs at the training learning rate); removing connections gradually while learning ends a further 0.9 points higher on both models at their tightest budget (97.8% and 85.7%), is level at 3% on the CNN, and costs 23–31% of the training compute of pruning the trained network gradually; on MNIST it never falls below 97% on the way (Figure 1).
3. **Honest cost accounting.** Prune-during costs 3–59× the cumulative training FLOPs of the dense small network because it starts large, and 26–45% of the cost of training the dense network it starts from, the usual train-then-compress baseline; RigL costs about the same as the dense small network on the MLP and CNN and 2.3–2.5× on ResNet-18. On CNNs, equal connection counts are not equal FLOPs; constraining the layer-wise allocation to the

ERK pattern, which uses fewer inference FLOPs than the dense small network, still leaves a 4.2-point advantage at 1%.

4. **Transfer from a pre-trained network** (Table 4). Pruning an ImageNet-pre-trained ResNet-18 *during* its adaptation to CIFAR-10 keeps 93.7% at 5% and 91.0% at 2% of its connections, against 88.4% and 87.5% from scratch and 86.9% for RigL from a random mask on the same weights; RigL from the inherited magnitude mask comes within 0.4 points at 28–42% of the adaptation compute. Under the standard schedule the advantage reverses at 0.5% (80.9% against 83.8% from scratch), but ending the schedule earlier or rewiring from the inherited mask restores parity with pruning from scratch (83.7% and 84.0% against 83.8%): the inherited start adds accuracy, not adaptation compute, and at 0.5% needs time at the final density to break even.
5. **Local pruning rules.** A Hebbian correlation rule beats random removal at 5% and below (not at 10%) but falls below the dense small network at every budget under 10%. A synapse-level rule (“synaptic drive”: the input flowing through a synapse when its post-synaptic unit fires) is competitive on the MLP but collapses on the CNN through channel death unless normalised per neuron, and trails magnitude pruning in both settings.
6. **Negative results** (Section 5) for a simple sleep-consolidation scheme with a fast hippocampal learner, for predictive token skipping (neither its choice of tokens nor its predicted-change substitution beats a random or identity control), for thalamic routing (pre-decided token skips no better than random; pre-routed sparse attention saving 3.6–4.7% of compute), and for input-dependent pruning at inference time (which forms input-specific subnetworks yet loses to a fixed mask at the same budget), together with the observation that STDP alone sparsifies a spiking network during learning.

2 Related work

Pruning. Magnitude pruning after training [13], gradual pruning during training with a cubic sparsity schedule [30], the lottery-ticket hypothesis [8] and data-free saliency criteria such as SynFlow [26] are the closest antecedents; Hoefler et al. [14] survey the field. Gale et al. [10] found at ImageNet and WMT scale that gradual magnitude pruning matches or beats variational dropout and ℓ_0 regularisation, and Liu et al. [17] that for structured pruning a network of the pruned architecture trained from scratch matches the fine-tuned pruned network, whereas for unstructured pruning at high sparsity on ImageNet it does not. Our dense small baseline is the width-scaled form of that from-scratch control, and our budgets (0.5–10%) lie on the high-sparsity side of their finding: it loses to prune-during by 0.3 points at 10% and by 3.7–8.6 points at the tightest budgets. Our prune-during arm is gradual magnitude pruning; our contribution is the controlled comparison at these budgets, the three-currency accounting, the local-rule variants and the transfer regime.

Dynamic sparse training. SET [21] and RigL [7] start sparse and periodically drop and regrow connections (randomly or by gradient magnitude), with the Erdős–Rényi-kernel (ERK) layer allocation (we use ERK on the convolutional networks and a uniform allocation on the MLP). RigL is the cheapest of the large-network routes and, at the tightest MNIST and ResNet-18 budgets, the strongest alternative to prune-during; SET, run on MNIST only, trails it. All RigL numbers are at the same training length as the other routes (15–20 epochs); Evci et al. [7] report that RigL needs 2–5× longer training to match or beat gradual magnitude pruning, so this setting favours pruning, and Section 4 adds 3× longer RigL runs.

Sparse attention and token skipping. Sparse Transformers [2], top- k attention [12] and routing attention [23] sparsify attention; DynamicViT [22] and A-ViT [29] drop tokens during a forward pass. Predictive coding [9, 20] motivates processing only prediction errors.

Complementary learning systems. The hippocampus–cortex account [16, 19] underlies replay-based continual learning: generative replay [25], CLS-ER [1], and class-balanced reservoir sampling [3]. Two implementations of the hippocampus–cortex structure do succeed at class-incremental learning without storing data: brain-inspired replay [28], in which the network’s own context-modulated feedback connections generate internal representations for replay, and CH-HNN [24], a hybrid of artificial and spiking networks that emulates the dual (specific and generalised) representations of corticohippocampal circuits and uses episode inference from prior knowledge. Our implementation in Section 5 is deliberately simpler—two plain MLPs, replay from a buffer, distillation—and its failure should be read against these successes. Synaptic homeostasis during sleep [27] motivated our downscaling variant. Pruning-based continual learning [11, 18] is related to the follow-up we propose in Section 6.

STDP. Our spiking network follows Diehl and Cook [4]. The accuracy ceiling of unsupervised STDP on shallow networks and its difficulty in scaling to deeper architectures are known [6]; the strongest STDP-only results add adaptive thresholds, adaptive synaptic filters and adaptive lateral inhibition [5]. We keep the plain rule because our interest is in what it does to the weight distribution, not in its accuracy.

3 Setup

Routes to a sparse network. Let B be a final budget of non-zero weights. We compare: *dense small*: a network of the same architecture whose width is scaled so that it has B weights, trained normally (additive); *prune-after*: the full-width network trained to completion, pruned once to B by global magnitude, then fine-tuned for half the original schedule; *prune-during*: the full-width network trained while a cubic schedule [30] lowers the density from 1 to $B/|W|$ between 10% and 70% of training, pruning every 100 steps with no regrowth; *SET* and *RigL*: a random mask at density $B/|W|$ (uniform per layer on the MLP, ERK on the CNN) with drop-and-regrow every 100 steps up to 75% of training, with random or gradient-based regrowth; *static sparse*: the random mask kept fixed. A full-width *dense big* network trained without pruning gives the accuracy ceiling.

Pruning criteria. Within prune-during we vary what is removed: weight magnitude $|w_{ij}|$ (global or per-layer threshold); a Hebbian correlation trace $|\mathbb{E}[y_i x_j]|$ with post-activation y_i and input x_j ; its product with magnitude; a *synaptic drive* trace $|w_{ij}| \mathbb{E}[\mathbb{1}(y_i > 0) |x_j|]$, i.e. the mean input carried by the synapse while its post-synaptic unit is active, which transports the “pre fires and post fires” condition of STDP to a synapse-level score without gradients; a per-neuron-normalised drive (dividing by the neuron’s mean drive, a homeostatic correction); and random removal as a control. All traces are exponential moving averages (momentum 0.99) maintained by forward hooks; convolutional drives are computed exactly per weight via unfolded receptive fields.

Models and data. MNIST: MLP 784–1024–1024–10 (1,861,632 weights), Adam, cosine schedule, 15 epochs, budgets 10%, 5%, 2%, 1% and 0.5%. CIFAR-10: VGG-style CNN with BatchNorm (conv 64-64/128-128/256-256, fc 256; 2,195,648 weights), SGD with Nesterov momentum, cosine schedule, 20 epochs, random crop and flip, budgets 10%, 3% and 1%. On the CNN a uniform per-layer

Table 1: MNIST test accuracy (%) at matched final connection budgets, 3 seeds. Standard deviations are below 0.1 for the magnitude routes at every budget and below 0.3 for every route at 2% and above; at 1% random removal and the static mask reach 0.6 and 0.3, and at 0.5% they reach 2.3 and 2.7, with prune-after 0.6, activity 1.1, RigL 0.3 and drive 0.2; full values in Table 7. Density is relative to the 1.86M-weight network. The dense big network reaches 98.7%. Prune-after, gradual applies the cubic schedule during the fine-tuning phase after dense training (same total steps as prune-after); RigL, 3 $\times$ trains for 45 epochs.

Route (criterion)	10%	5%	2%	1%	0.5%
Dense small (additive)	98.36	98.01	97.30	96.30	94.07
Prune-after + fine-tune	98.69	98.73	98.38	97.52	94.51
Prune-after, gradual + fine-tune	98.70	98.64	98.49	97.94	96.92
Prune-during, magnitude (global)	98.67	98.72	98.65	98.44	97.81
Prune-during, magnitude (uniform per layer)	98.67	98.68	98.55	98.23	97.04
Prune-during, synaptic drive	98.53	98.50	98.22	97.56	96.03
Prune-during, $ w \times$ activity (Hebbian)	98.57	98.42	98.05	97.25	95.22
Prune-during, activity (Hebbian)	98.19	97.87	96.55	93.66	88.85
Prune-during, random	98.23	97.13	93.39	89.13	71.84
RigL	98.53	98.22	97.85	97.41	96.31
RigL, 3 $\times$ training length	—	—	—	97.72	97.46
SET	98.38	98.19	97.43	96.03	93.65
Static random sparse	98.18	97.54	95.90	92.59	72.71

density starves the first convolution (1,728 weights), so per-layer criteria use the ERK allocation as their layer targets. Three seeds for the MNIST, CIFAR-10 CNN and ResNet-18 comparisons; two for the transfer experiment, the inference-time pruning arms, the 20% ResNet-18 rows and the uniform-allocation ResNet-18 control; one for the CIFAR-10 ViT runs and for the 100- and 1,600-neuron spiking runs, which we mark as such. We report mean $\pm$ standard deviation.

Metrics. Test accuracy; inference FLOPs counted analytically as $2\sum_{\ell} n_{\ell}d_{\ell}$ over linear and convolutional layers with n_{ℓ} active weights and d_{ℓ} output positions; cumulative training FLOPs as three times the forward FLOPs of the *actual* active network at every step; expected calibration error; data efficiency as the number of training samples needed to reach a target accuracy; and, for the sequential-task experiment, average final accuracy and forgetting. We do not measure energy: all experiments run on von Neumann GPUs (a consumer RTX 3060 Ti with 8 GB; the ResNet-18, transfer and inference-time pruning sweeps on one rented A40), where the brain’s energy argument cannot be tested.

4 Prune-during versus the alternatives

Accuracy at matched budgets. Tables 1 and 2 give the main result. Prune-during with a global magnitude criterion is the most accurate route at every budget of 2% and below. At the loosest budgets prune-after is level with it or slightly ahead: by 0.02 points at 10% and 0.01 at 5% on MNIST, 0.4 at 10% on CIFAR-10 and 0.3 at 5% on ResNet-18 (Table 3). Its margin over the additive baseline grows as the budget shrinks: on MNIST from 0.3 points at 10% to 3.7 points at 0.5%, on CIFAR-10 from 3.0 points at 10% to 8.6 points at 1%. It also beats RigL by 1.5 (MNIST, 0.5%) and 9.8 points (CIFAR-10, 1%, against the two RigL seeds that converged), and prune-after by 3.3 and 4.6 points (the CNN figure is against prune-after fine-tuned at the training

Table 2: CIFAR-10 test accuracy (%) and inference FLOPs per image at matched budgets (3 seeds). Density is relative to the 2.2M-weight CNN, which reaches 91.3% dense. Per-layer criteria use the ERK allocation. RigL at 1%: one of three seeds diverged to chance despite the non-finite-gradient guard (Appendix); the table shows the mean of the two seeds that converged, and the three-seed mean is 53.9 ± 31.1 (Table 8, Figure 2); the same happened to one seed of the $3 \times$ run (three-seed mean 56.7 ± 33.0). Prune-after, gradual applies the cubic schedule during the fine-tuning phase after dense training (same total steps as prune-after); dense small, shallow and wide has four convolutional layers instead of six at the same weight count; RigL, $3 \times$ trains for 60 epochs. Prune-after rows fine-tune at lr 0.01, the standard protocol (Appendix A), unless marked lr 0.05, a cosine restart at the training rate.

Route (criterion)	Accuracy			Inference FLOPs		
	10%	3%	1%	10%	3%	1%
Dense small (additive)	87.3	82.8	77.1	3.1e7	9.7e6	3.3e6
Dense small, shallow and wide	86.9	83.9	78.4	8.1e7	2.6e7	8.5e6
Prune-after + fine-tune	90.7	88.4	80.5	8.2e7	3.6e7	1.6e7
Prune-after, gradual + fine-tune	90.5	88.0	81.9	8.1e7	3.4e7	1.5e7
Prune-after + fine-tune at lr 0.05	–	88.4	81.1	–	3.6e7	1.6e7
Prune-after, gradual, fine-tune at lr 0.05	–	89.2	84.8	–	3.1e7	1.3e7
Prune-during, magnitude (global)	90.3	89.1	85.7	7.4e7	3.0e7	1.2e7
Prune-during, magnitude (ERK)	89.3	86.9	81.3	2.7e7	8.4e6	2.8e6
Prune-during, drive, per-neuron normalised (ERK)	87.6	79.3	60.0	2.7e7	8.4e6	2.8e6
RigL (ERK)	87.6	83.5	75.9 [†]	2.7e7	8.4e6	2.8e6
RigL (ERK), $3 \times$ training length	–	87.0	80.0 [†]	–	8.4e6	2.8e6

rate, the stronger variant; under the standard lower fine-tuning rate it is 5.2). The additive baseline is the width-scaled network; a depth variant with four convolutional layers instead of six at the same weight count (wider stages, $2.6 \times$ the inference FLOPs) does 1.1–1.3 points better than the width-scaled one at 3% and 1% (83.9% and 78.4%) and 0.4 points worse at 10%, still 3.4–7.3 points behind prune-during (Table 2); the claim that learning small loses is about these two shapes of small network, not about every small architecture.

The benefit is mostly in the gradual schedule; pruning while learning adds up to a point.

Figure 1 shows why prune-after loses at high sparsity: the one-shot removal destroys the function and fine-tuning recovers only part of it. Gradual removal interleaved with learning never leaves the high-accuracy region. The same picture holds on CIFAR-10 at 1% (91.4% before the pruning step, 10% after it, 80.5% after fine-tuning; run logs). A control separates the two ingredients, gradualness and timing: pruning the trained network gradually, with the same cubic schedule spread over the fine-tuning phase and the same total steps as prune-after, reaches 96.9% at 0.5% on MNIST against 94.5% for one-shot removal and 97.8% for prune-during, and 97.9% against 97.5% and 98.4% at 1% (Table 1). On MNIST the schedule accounts for 73% of the gap at 0.5% (2.4 of 3.3 points) and about half at 1% (0.4 of 0.9), interleaving removal with learning for the rest. On the CNN the same control first looked different: under the standard prune-after protocol, which fine-tunes at a fifth of the training learning rate (0.01 against 0.05; Appendix A), gradual prune-after matched one-shot prune-after at 10% and 3% (90.5% against 90.7% and 88.0% against 88.4%) and gained only 1.4 points at 1% (81.9% against 80.5%), 3.8 points behind prune-during (85.7%). That gap was the learning rate, not the timing: with fine-tuning restarted at the training rate (0.05, cosine), gradual prune-after reaches 89.2% at 3% and 84.8% at 1%, level with prune-during at 3% (89.1%)

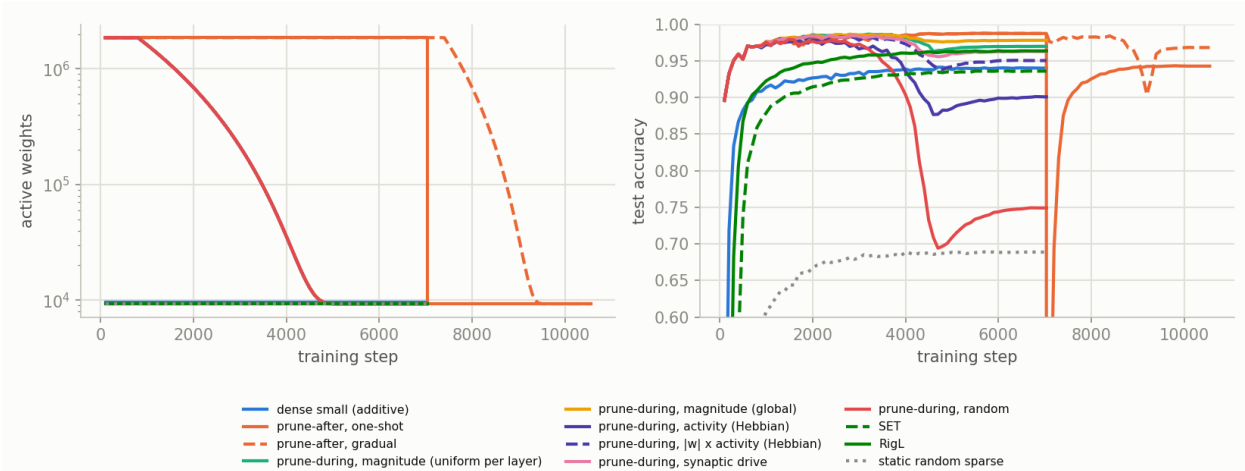


Figure 1: Connection count (left, log scale) and test accuracy (right) during training at 0.5% density on MNIST (seed 0). Prune-after (orange, solid) trains dense, is pruned once at step 7035, collapses below 60% and recovers to 94% during fine-tuning; gradual prune-after (orange, dashed) trains dense and then prunes along the cubic schedule during fine-tuning, ending at 96.8%; the prune-during routes reach the same final count gradually; the global route (yellow) never falls below 97% (lowest point in any seed 97.3%, run logs) and the per-layer route (green) ends at 97.0%.

and 0.9 points behind at 1%, while one-shot prune-after stays at 88.4% and 81.1% (Table 2). The decomposition is then the same on both models: the gradual schedule carries most of the benefit, and removing connections while the function is still being learned adds 0.5–0.9 points at the tightest budgets, at 28% of the training cost of pruning the finished network gradually ($3.1e14$ against $1.1e15$ FLOPs at 1%). Sparsity itself is not the point: on MNIST the two routes that impose a random pattern, a static random mask and random removal during training, are the worst at every budget below 10% (72.7% and 71.8% at 0.5%).

Connection counts versus FLOPs. On the CNN, global magnitude pruning keeps convolutional weights, which are each used at many positions, and removes fully connected ones, so the pruned network uses $2.4\text{--}3.5\times$ the inference FLOPs of the dense small network at the same connection count (Table 2). Constraining the per-layer allocation to ERK, which on this CNN gives *fewer* inference FLOPs than the dense small network, still leaves prune-during 4.2 points ahead at 1%, and 1.7–3.4 points ahead of RigL at the same FLOPs at 10% and 3% (5.4 at 1% against the converged RigL seeds, Table 2; Figure 2). On ResNet-18 the ERK variant costs $2.3\text{--}2.5\times$ the inference FLOPs of the dense small network, so there the equal-FLOPs comparison holds only against RigL (Table 3). The gain is therefore real in both currencies, but a comparison that matches connection counts alone overstates the compute equivalence of the two networks by a factor of $2.4\text{--}3.5$: at equal connections the pruned CNN spends that much more inference compute than the dense small one. The shallow-and-wide small network of Table 2 closes this gap from the other side: at the same connection count its inference FLOPs are within 10–30% of global prune-during’s ($8.1e7$ against $7.4e7$ at 10%, $2.6e7$ against $3.0e7$ at 3%, $8.5e6$ against $1.2e7$ at 1%), so the two are matched in both currencies, and it still trails by 3.4–7.3 points.

Training cost, data efficiency, calibration. Prune-during starts with the full network, so its cumulative training FLOPs are $3\times$ (10%) to $50\times$ (0.5%) those of the dense small network on MNIST,

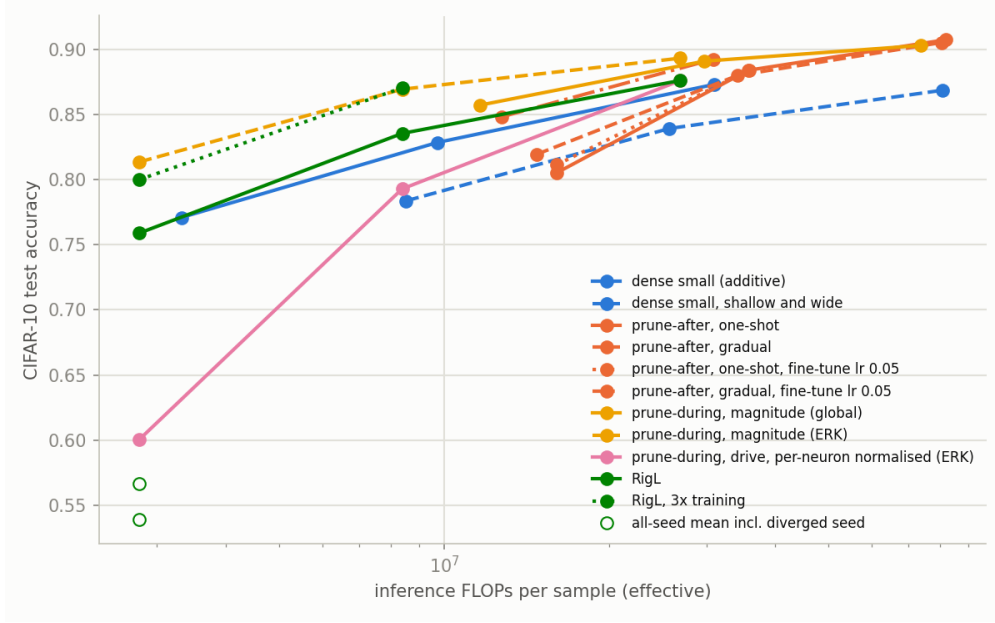


Figure 2: CIFAR-10 accuracy against inference FLOPs; each point is one budget. At equal FLOPs and equal training length, prune-during with the ERK allocation (yellow, dashed) dominates RigL (dark green) and the dense small network (blue); RigL trained three times longer (dark green, dotted) reaches the ERK line at 3%. Orange: prune-after, one-shot (solid) and gradual (dashed) at the standard fine-tuning rate and both at the training rate (dotted, dash-dotted); blue dashed: the shallow-and-wide dense small network. RigL at 1% is plotted as the mean of the two seeds that converged (75.9%), as in Table 2; the hollow markers are three-seed means that include a diverged seed (53.9% for RigL, 56.7% for RigL at $3\times$ training length).

4.5–31 $\times$ on CIFAR-10 and 8–59 $\times$ on ResNet-18; prune-after costs a further 2.5–4 $\times$. RigL costs about the same as the dense small network on the MLP and CNN (as does SET, run on MNIST only) and 2.3–2.5 $\times$ on ResNet-18. Against the usual industrial baseline, training the dense network and compressing it afterwards, prune-during is the cheaper route: 26–33% of the dense network’s training cost on MNIST (2.6–3.3e12 against 1.0e13 FLOPs), 33–45% on the CNN (3.1–4.2e14 against 9.2e14) and 33–40% on ResNet-18 (1.1–1.3e15 against 3.3e15), because the network shrinks while it trains, whereas prune-after costs the dense training plus fine-tuning. On the training-cost axis RigL is the most efficient of the large-network routes; in accuracy it is second to prune-during only at the tightest budgets (0.5% on MNIST and ResNet-18), otherwise third behind prune-after, and at 1% on CIFAR-10 it falls below the dense small network. Because RigL is known to close part of its gap with longer training [7], we also ran it for three times the epochs on MNIST (45 epochs; Table 1): at 1% it rises from 97.4% to 97.7% and at 0.5% from 96.3% to 97.5%, still 0.7 and 0.3 points behind global prune-during trained for the standard length, at 6–11% of prune-during’s training cost. On the CNN (60 epochs) the longer run gains more, from 83.5% to 87.0% at 3% and from 75.9% to 80.0% at 1% (converged seeds; one seed of three diverged again), but stays 2.1 and 5.7 points behind global prune-during at 8–22% of its training cost (Table 2). Part of that remaining gap is the allocation, not the timing: RigL keeps a uniform (MLP) or ERK (CNN) layer allocation, and against prune-during under the same allocation the $3\times$ run is level or ahead at two of the four budgets (MNIST per-layer prune-during 98.2% and 97.0% at 1% and 0.5% against 97.7% and 97.5%; CNN ERK prune-during 86.9% and 81.3% at 3% and 1% against 87.0% and

Table 3: CIFAR-10 ResNet-18 (11.2M weights, 92.5% dense): test accuracy (%) and inference FLOPs per image at matched budgets, three seeds. Per-layer criteria use the ERK allocation.

Route (criterion)	Accuracy			Inference FLOPs		
	5%	2%	0.5%	5%	2%	0.5%
Dense small (additive)	88.7	86.5	81.3	5.6e7	2.3e7	6.1e6
Prune-after + fine-tune	92.3	90.9	83.6	2.7e8	1.5e8	5.3e7
Prune-during, magnitude (global)	92.0	91.5	89.0	1.9e8	1.1e8	3.4e7
Prune-during, magnitude (ERK)	92.0	91.3	87.8	1.4e8	5.7e7	1.4e7
RigL (ERK)	91.0	89.5	84.9	1.4e8	5.7e7	1.4e7

80.0%). Prune-during’s advantage over dynamic sparse training therefore rests on two things it is free to do, remove gradually during learning and choose the allocation globally; given three times the training, RigL largely closes the first but not the second. Measured in samples rather than FLOPs, the over-parameterised network learns fastest, but on MNIST it reaches 97% after 87k samples, before pruning begins, so this is a property of the dense start that every large-network route shares (against 133k–436k samples for the dense small networks and 124k–445k for RigL). The comparison is informative where the target falls inside the pruning phase: on ResNet-18 at 5% prune-during reaches 85% after 385k samples, when about a fifth of the weights remain, earlier than the unpruned dense network (437k; prune-after is still dense at that point), RigL (524k) and the dense small network (665k); at 0.5% it needs 402k, against 437k, 908k (one RigL seed of three, the others never) and never. On MNIST the expected calibration error stays between 0.003 and 0.017 for the magnitude, additive, prune-after and RigL routes at every budget and degrades for the local rules at the tightest budgets (drive 0.025, $|w| \times \text{activity}$ 0.038, activity 0.100), for the static random mask (0.053) and for random removal (0.157); on the CNN and ResNet-18 all routes lie between 0.008 and 0.038.

ResNet-18. Table 3 repeats the comparison with a CIFAR-style ResNet-18 (11.2M prunable weights, 92.5% dense; SGD, 20 epochs, three seeds) at 5%, 2% and 0.5% of the weights. The winner is unchanged: at 0.5% (56k weights) prune-during reaches 89.0% against 81.3% for the dense small network, 83.6% for prune-after and 84.9% for RigL, its largest lead over prune-after in any setting (5.3 points) and a lead over the additive baseline (7.7) close to the CNN’s (8.6); constrained to the ERK allocation it keeps 87.8% at 40% of the inference FLOPs of the global variant. The order of the other routes does change with the model: on the CNN at 1% prune-after beats dense small beats RigL, on ResNet-18 at 0.5% RigL beats prune-after beats dense small. At 5% the three magnitude routes (prune-after, global and ERK prune-during) lie within 0.3 points of each other and 0.2–0.5 points below the dense ceiling (92.5%), with RigL a point behind them, so at loose budgets the budget, not the method, decides whether the route matters.

What to remove: local rules. Removing connections by a Hebbian correlation trace beats random removal at 5% and below (not at 10%) but falls below the dense small network at every budget under 10% (88.9% against 94.1% at 0.5%), because $\mathbb{E}[y_i x_j]$ measures correlation through every path, not the contribution of synapse ij . Scoring synapses by their own drive repairs most of the gap on the MLP: at 0.5% the drive rule (96.0%) beats the additive baseline, prune-after and SET and is 0.3 points behind RigL, with a criterion that uses no gradient information (training itself remains back-propagation). On the CNN the unnormalised drive rule collapses within two epochs (run logs): channels that fire rarely lose all their inputs, die, and the failure propagates

downstream, a known hazard of activity-dependent pruning without homeostasis. Normalising each neuron’s drive by its mean removes the collapse but the rule still falls from parity with the dense baseline at 10% to 17 points behind it at 1%. A local criterion can stand in for the magnitude criterion on the MLP at a cost of 1.0 point at 0.5% against the per-layer magnitude criterion (1.8 against the global one); on the convolutional network, in this form, it cannot.

Transfer: pruning a pre-trained network during adaptation. The over-parameterised starting point that prune-during needs is, in practice, often inherited: pre-trained networks are published with their training cost already paid, so the practical question is what gradual pruning does during adaptation. We therefore took an ImageNet-pre-trained ResNet-18 (torchvision weights, 11.17M convolutional weights, fresh 10-way classifier kept dense) and adapted it to CIFAR-10 for 10 epochs at 128×128 input while pruning its convolutions to 5%, 2% and 0.5% with the same cubic schedule (SGD, lr 0.01, the from-scratch arms use lr 0.1 and the same architecture, schedule and resolution). Table 4 compares gradual pruning during adaptation with one-shot magnitude pruning of the same weights followed by fine-tuning, RigL started either from a random ERK mask over the pre-trained weights or from the pre-trained magnitude mask at the target density (the fair form, which keeps the inherited structure and rewires it), and the two from-scratch routes. At 5% and 2% the pre-trained start is worth 5.3 and 3.5 points over pruning from scratch, gradual removal during adaptation beats one-shot pruning of the same weights by 0.6–0.7 points, and RigL from a random mask, which discards the inherited structure, lands below pruning from scratch; RigL from the magnitude mask, however, comes within 0.3–0.4 points of gradual pruning (93.3% and 90.7%, two seeds) at 28–42% of its adaptation compute, so once the inherited structure is kept, how the sparse network is then reached matters little at these budgets. The pre-trained prune-during arms reach 90% after about 52k samples (just over one epoch, when pruning has just begun and about 99% of the weights remain, as does the dense fine-tune), a level one-shot pruning needs 166k–402k samples for at 5% and 2% and does not reach at 0.5%, and RigL never reaches in ten epochs; they are also better calibrated (ECE 0.015–0.020 against 0.028–0.031 for prune-during from scratch). The inherited start does not save adaptation compute: at 5% gradual pruning costs $7.3e14$ FLOPs from the pre-trained weights against $7.7e14$ from scratch, 40% of a dense fine-tune and $7\times$ the dense small network. What it buys is accuracy; the early data efficiency is that of the dense start, which gradual pruning keeps and one-shot pruning destroys. At 0.5% the standard schedule reverses the ordering: pruning from scratch (83.8%) beats one-shot pruning (81.7%), which beats gradual pruning of the pre-trained network (80.9%). The pre-trained prune-during run peaks at 93.1% in mid-schedule and loses twelve points as the last 4.5% of its connections go, with only three epochs left at the final density, whereas one-shot pruning spends all ten there. Two controls show that what is missing is time at the final density for a network whose connections are still being chosen, not the inherited structure as such; one-shot pruning spends all ten epochs at 0.5% but with a mask fixed at the start and reaches only 81.7%. Ending the schedule at 50% of adaptation instead of 70%, which leaves five epochs at 0.5%, lifts gradual pruning to 83.7% (two seeds), level with pruning from scratch and above one-shot pruning at 82% of the compute, and RigL from the magnitude mask, which rewires for all ten epochs at the final density, reaches 84.0% at a seventh of the compute. An inherited structure therefore pays off down to 2% and, given enough adaptation time at the final density, breaks even at 0.5%; the practical choice there is between a schedule that ends early and dynamic sparse training from the inherited mask. The latter also isolates one more ingredient: one-shot pruning and magnitude-mask RigL share the start, the mask and the cost and differ only in whether the mask is rewired, and rewiring is worth 2.3 points at 0.5% (two seeds) and 0.2–0.4 points at 5% and 2%, the latter comparable to the seed spread.

Table 4: ImageNet-pre-trained ResNet-18 adapted to CIFAR-10 (128×128 , 10 epochs) while being pruned: test accuracy (%) at matched convolutional budgets, two seeds, standard deviations ≤ 0.4 . Adaptation FLOPs count the 10 epochs only; the ImageNet pre-training is a sunk cost common to the pre-trained arms. The unpruned dense fine-tune reaches 95.9% at 1.8×10^{15} FLOPs. The schedule that ends at 50% of adaptation was run at 0.5% only (Table 10).

Start	Route	5%	2%	0.5%	Adapt. FLOPs (5%)
pre-trained	prune-during, magnitude (global)	93.7	91.0	80.9	7.3e14
pre-trained	prune-during, global, schedule ends at 50%	–	–	83.7	–
pre-trained	prune-during, magnitude (ERK)	93.4	89.7	78.8	8.1e14
pre-trained	one-shot prune + fine-tune	93.1	90.3	81.7	3.0e14
pre-trained	random ERK mask + RigL	86.9	83.6	76.8	2.7e14
pre-trained	magnitude mask + RigL	93.3	90.7	84.0	3.0e14
scratch	prune-during, magnitude (global)	88.4	87.5	83.8	7.7e14
scratch	dense small (additive)	84.8	82.4	76.5	1.1e14

5 Negative results from the same programme

The programme that produced Section 4 also implemented the other mechanisms of Table 5. Four of them did not work as designed, and we report them because the failures are informative: sleep consolidation, thalamic routing, predictive coding and per-input subnetworks. The local pruning criteria of Section 4 are a variant of the first mechanism and are not counted separately; the spiking network closes the section as a replication, not a failure.

Sleep consolidation. We trained a fast “hippocampal” MLP on the task stream and a slow “cortical” MLP only during sleep phases at task boundaries, using the current task’s data plus a 500-sample reservoir buffer, with distillation from the fast network, global synaptic downscaling and magnitude or activity pruning as optional sleep ingredients (Split MNIST class-incremental, Permuted MNIST, MLP 784–256–256–10, three seeds; Tables 11 and 12). Distillation from the fast network lowered final average accuracy on Split MNIST from 85.0% to 78.7% and raised forgetting from 0.18 to 0.25, because the fast network, trained on the current task’s classes alone under the same single 10-way head, is biased to them; downscaling hurt in both settings (77.4% and 82.7%); magnitude pruning was neutral and activity pruning cost 6.3 points on Permuted MNIST; and the scheme without distillation only matched plain experience replay (85.0% against 85.2%) at three times the compute. A redesign with a k -winners-take-all sparse hippocampus, no distillation and class-balanced replay improved to 87.0%, but balanced sampling alone added to experience replay reaches 87.8%, and the hippocampal sparsity has no path to influence the cortex once distillation is removed: the cortex’s numbers are identical to four decimals whether the hippocampus is dense or k -WTA at 10% or 5% (87.03 ± 0.59 in all three rows of Table 11), because the cortex is trained only on buffered data and labels, which the hippocampus does not touch. On Split MNIST the only effective ingredient was the known one [3]; on Permuted MNIST the scheme without distillation beats plain experience replay by 1.2 points at the same compute (88.6% against 87.4%), but the fair comparison is class-balanced replay, which reaches 88.3% at 40% of that compute, leaving 0.3 points at $2.5\times$ the compute; the sparse redesign beats balanced replay by 0.8 points at five times the compute (89.0% against 88.3%). This is a negative result for our simple implementation, not for the hippocampus–cortex account: brain-inspired generative replay [28] and the hybrid CH-HNN [24] both succeed at class-incremental learning, and what they have that we lacked is a hippocampus with generative capacity (internal representations replayed through feedback connections) or a dual

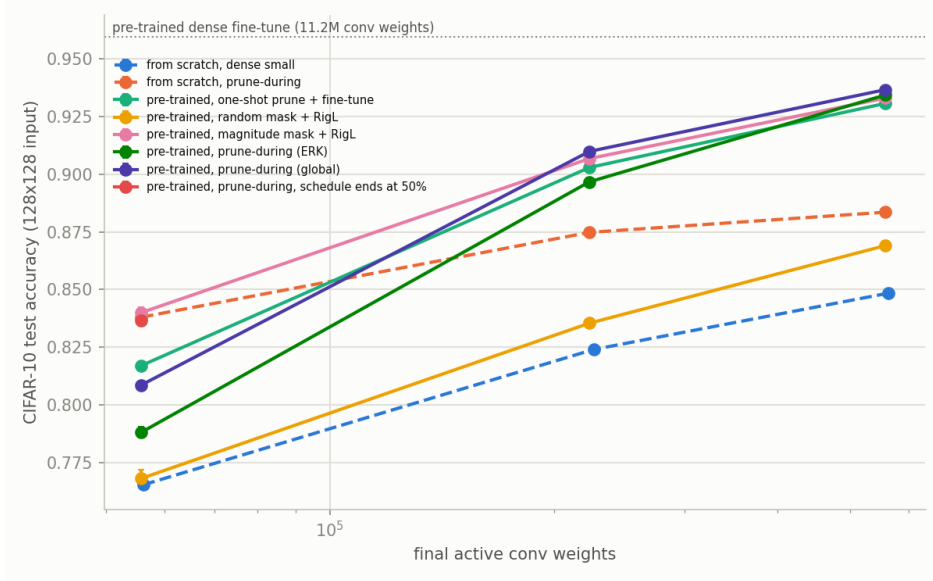


Figure 3: Pre-trained ResNet-18 adapted to CIFAR-10 while pruned: accuracy against final active convolutional weights (two seeds). Solid lines start from the ImageNet weights, dashed lines from scratch. The pre-trained advantage is largest at 5%; at 0.5% only the routes that leave time at the final density, the magnitude-mask RigL and the early-ending schedule, reach parity with pruning from scratch.

specific-generalised representation with episode inference, rather than a second feed-forward network that can only speak to the cortex through distillation. Confidence-based agreement between the two networks, proposed as a metacognitive “I do not know” signal, failed to detect unseen classes (AUROC 0.51), whereas plain softmax confidence reached 0.85. Varying how the sleep phase decays weights, on the sparse redesign (Tables 19 and 20), did not change the picture: a one-shot 10% downscaling at the task boundary, a gradual 10% or 3% decay spread over the sleep phase and a continuous weight decay were within noise on Split MNIST (+0.1 to +0.5 points) and cost 0.3–4.0 points on Permuted MNIST. Replacing global logit distillation by layer-local feature distillation, the form a processor-memory split would favour, did not help: with a 256-unit fast network, local distillation costs 4.2 points against no distillation on both benchmarks, less than global distillation on Split MNIST (6.2) and more on Permuted MNIST (1.6).

Predictive token skipping and pre-filtering. With a low-rank predictor per block estimating how much each token will change, skipping the tokens predicted to change least, uniformly over all blocks of a small ViT, was *worse* than skipping random tokens (MNIST, 30% skip: 94.9% against 96.7%; even the oracle residual norm was worse than random), because background tokens freeze consistently in every layer. Restricting skipping to the later half of the blocks recovers most of the loss by itself: with the skipped tokens left unchanged and no fine-tuning, skipping 70% of them in the late blocks costs 0.3 points on MNIST and 5.0 on CIFAR-10, against 1.5–3.3 and 8.9–15.4 points for skipping 30% of them in every block at slightly higher FLOPs (0.77 against 0.72 of the full model; Tables 13 and 14), and a short fine-tuning phase that ramps the skip ratio from 0 to 0.7 brings the CIFAR-10 cost to 1.0 point at 72% of the FLOPs (ViT, 81.8% baseline, one seed; Figure 4). Replacing the skipped tokens by their predicted residual, the predictive-coding ingredient, adds nothing over leaving them unchanged under the same fine-tuning: 97.9% against 97.9% on MNIST

Table 5: The mechanisms ported in this paper, their reductions, and what transferred.

Brain mechanism	Our reduction	Outcome	Where
gradual synaptic pruning during learning	prune-during at matched budgets	transfers (the principle; a schedule-level change)	Section 4
activity-dependent local pruning rules	Hebbian, drive and normalised-drive criteria	mixed: drive about a point behind per-layer magnitude pruning on the MLP, Hebbian eight points behind; the drive rules collapse on the CNN without homeostasis	Section 4
hippocampus-cortex sleep consolidation	fast/slow MLPs, replay, distillation, decay	does not transfer: within 0.3–0.8 points of class-balanced replay at 2.5–5× its compute	this section
thalamic routing	pre-decided skips; pre-routed attention	does not transfer (choice uninformative; little compute saved)	this section
predictive coding	skip by predicted change; predicted-residual substitution	does not transfer: selection no better than random, substitution no better than identity	this section
per-input subnetworks at inference (follow-up)	per-sample masks by a local rule	does not transfer	this section
spike-timing-dependent plasticity	Diehl–Cook network	mixed: known accuracy limit reproduced; self-sparsification observed	this section

(three seeds) and 80.8% against 80.8% on CIFAR-10 at 70% skipping (one seed; within 0.7 points at every non-zero ratio, with a 1.0-point gap at zero skipping that sets the single-seed noise floor; Tables 15 and 16). A “thalamic” router that pre-decides all late-block skips from the representation at the block boundary is indistinguishable from per-block prediction and from random selection, also under cross-evaluation (three seeds on MNIST; one seed on CIFAR-10, where the 1.0-point gap at zero skipping sets the noise floor; Tables 15 and 16). The flatness is not because the late blocks are idle: removing them altogether and fine-tuning for the same epochs costs 1.7 points on MNIST (96.6% against 98.3%, three seeds) and 9.0 points on CIFAR-10 (72.9% against 81.8%) at 50% of the FLOPs, far more than skipping 90% of their tokens (0.4 and about 2 points at 64–65% of the FLOPs; Tables 15 and 16). What the late blocks contribute passes through the always-computed class token and the few tokens kept, which is why which tokens are kept does not matter. The choice of *which* tokens to skip carries no information at this scale, and neither does what replaces them; what matters is how many are processed, in which layers, and that the late blocks run at all.

Pre-routed sparse attention. Selecting the top- k keys per query with an 8-dimensional router *before* the full attention computation keeps accuracy at $k = 5$ of 50 keys on MNIST (98.1% against 97.9%, three seeds) and $k = 17$ of 65 on CIFAR-10 (80.0% against 79.4%, one seed; Tables 17 and 18) and cuts attention FLOPs by 68–77%, but attention is only 5–6% of the total cost at 50–65 tokens, so total FLOPs fall by only 3.6–4.7%; post-hoc top- k masking still computes the full score matrix and saves at most 2.7%. Stacked on token skipping it saves a further 1% of FLOPs for a 0.1–0.6 point loss. The mechanism is sound but irrelevant below several hundred tokens.

Input-dependent pruning at inference time. If learning is pruning, should inference prune as well? We let the ResNet-18 of Table 3 select a different subnetwork for every input. For each output neuron, the connections with the largest $|w_{ij}| |\overline{x_j}|$ (the weight times the mean magnitude of its input channel for that sample, a per-sample form of the synaptic-drive rule without its post-synaptic term) are kept and the rest are masked for that sample only, with the kept fraction following the

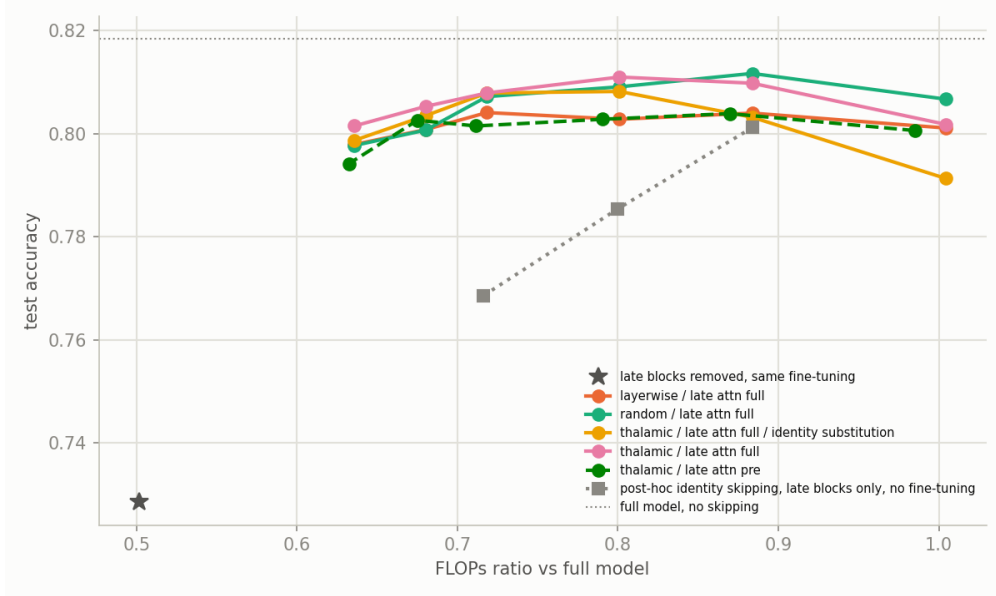


Figure 4: CIFAR-10 ViT: accuracy against FLOPs when skipping tokens in the late blocks with predicted-residual substitution, after fine-tuning with a ramped skip ratio (one seed). Thalamic pre-decision, per-block prediction, random selection and identity substitution (no predicted residual) are indistinguishable at single-seed noise; all lie far above post-hoc identity skipping without fine-tuning (grey).

same cubic schedule as prune-during so that every sample uses the budget (within 1% at 20% and 5%; 14% more at 0.5%, where per-neuron rounding and the dense first layer dominate). Controls: the same per-sample budget chosen at random, a channel-level k -WTA gate on each convolution’s input (the only variant whose sparsity is structured enough to save time on a GPU), and the static networks of Table 3 plus a static mask with the same uniform per-layer allocation that the dynamic rule is confined to. Table 6 gives the result. At 20% the input-dependent rule matches the fixed masks (91.8% against 91.8–91.9%). The penalty appears as the budget shrinks: at 5% it reaches 89.8% against 92.0% for a fixed global mask and 91.1% for a fixed mask with the same per-layer allocation, and at 0.5% 77.7% against 89.0% and 83.1%, so re-deciding the mask for every input costs 1.3 points at 5% and 5.4 points at 0.5% even after allocation is controlled for; it beats the width-matched dense network at 20% and 5% (90.9%, 88.7%) and loses to it at 0.5% (81.3%). In FLOPs the dynamic arm sits between the two fixed masks (at 0.5%, 9.3e6 against 5.6e6 for the uniform and 3.4e7 for the global mask). Random per-input selection collapses to 52.3% at 5% and 18.2% at 0.5%, and the channel-level gate to 44.9% and 34.0% (73.4% even at 20%). The rule does form input-specific subnetworks: in the last stage, the masks of two samples of the same class share 44% of their connections at 5% and those of different classes 27% (random masks: 2.6%; at 20%, 61% against 46%), and across 200 test images the network touches 21% of that stage’s connections while each image uses 5%. The extra capacity is real and buys nothing. This is the mirror image of the token result: among tokens, which to drop did not matter; among connections, which to keep is everything, and re-deciding it for every input is worse than deciding it once.

STDP sparsifies itself. A 400-neuron spiking network in the style of Diehl and Cook [4], trained with trace-based STDP, lateral inhibition and weight normalisation on one pass of MNIST, reaches

Table 6: Input-dependent (per-sample) pruning of a CIFAR-10 ResNet-18 against fixed masks at the same per-sample budget. Static rows from Table 3 (three seeds; the 20% column and the uniform-allocation control from Table 9, two seeds); dynamic rows two seeds, with per-sample budgets within 1% of the fixed masks at 20% and 5% and 14% above them at 0.5%. Jaccard: overlap of the last-stage masks of two test images of the same class / of different classes (200 images, 5% budget); a random mask gives 0.026.

Mask	20%	5%	0.5%	Jaccard same / diff. class (5%)
fixed, prune-during, magnitude (global)	91.9	92.0	89.0	–
fixed, prune-during, magnitude (uniform per layer)	91.8	91.1	83.1	–
fixed, dense small (additive)	90.9	88.7	81.3	–
per sample, connection-level, local rule	91.8	89.8	77.7	0.44 / 0.27
per sample, connection-level, random	76.1	52.3	18.2	0.03 / 0.03
per sample, channel-level k -WTA gate	73.4	44.9	34.0	0.08 / 0.05

81.1±0.3% (a back-propagated MLP of the same width reaches 96.5% on the same samples). The gap is the known limitation of unsupervised STDP [5, 6]; what we add is an observation about the weights. Without any explicit pruning, the fraction of its input weights below 1% of the maximum rises from 5.0% to 57.7% and the Gini coefficient of the weight distribution from 0.33 to 0.77 during learning (run logs; the 100- and 1,600-neuron runs end at 54.9% and 28.7%, Table 22). Much of this is prototype formation rather than pruning in the usual sense: with column-normalised weights and lateral inhibition each neuron converges to a digit template, and the pixels that are background for that template lose their weight. A control bounds that part: in the class-mean MNIST images 49.8% of the pixels (42–70% by class) fall below 1% of the maximum, and 37% of the pixels are zero in 99% of the training images (Table 22), so a 57.7% small-weight fraction is about what receptive fields that converge onto class templates would produce, with little beyond it. The 1,600-neuron run, less sparse (28.7%) and less accurate (72.9%) than the 400-neuron run after the same single pass, is under-trained rather than different, which points the same way: the sparsity is a function of learning progress. Learning under a local plasticity rule with normalisation is subtractive in this limited sense, emptying the weights that the template does not use, which is the biological reading of Section 4 at the level of a single layer.

6 Discussion and limitations

Scope of the claim. The claim that survives is narrow and, we think, useful: at a tight final connection budget, removing connections gradually while learning, with a global magnitude criterion, is better than learning small, better than removing them afterwards in one shot, and better than rewiring from a sparse start at equal training length, part of that last margin being the global allocation that RigL lacks. Against removing them gradually afterwards it gains only 0.5–0.9 points at the tightest budgets and is level at 3% on the CNN, but costs 23–31% as much to train, so for that comparison the case for pruning while learning is mainly one of cost. The advantage grows with sparsity, and at loose budgets it shrinks to a tie with removing them afterwards. The claim is about accuracy per connection, measured against small networks that are width-scaled or shallower-and-wider versions of the same architecture. It is not a claim about training cost, where RigL wins, and on convolutional networks it must be stated per FLOP as well as per connection. It is also not a claim that local rules suffice: the best of them ends about a point behind per-layer magnitude pruning on the MLP, and the drive rules we ran on the CNN fail there.

Training cost, and what an inherited start buys. The $3\text{--}59\times$ training-cost multiplier over the dense small network is the honest weakness of prune-during trained from scratch (though it is still 26–45% of the cost of training the dense network it starts from): it buys accuracy with compute, and RigL buys most of the accuracy for the price of the dense small network ($2.3\text{--}2.5\times$ on ResNet-18), trailing prune-during by 1.5–10 points at the tightest budgets (0.3–5.7 points when trained three times longer). Two observations qualify the weakness. First, training happens once and inference many times; at equal inference FLOPs (the ERK-constrained variant) the extra training compute is amortised over deployment, so the trade-off depends on how often the model is used. Second, the over-parameterised starting point is often inherited: pre-trained networks are published with their training cost already paid. That does not make the adaptation cheap, since the schedule still runs its dense phase (at 5%, $7.3e14$ FLOPs against $7.7e14$ from scratch), but it changes what the compute buys. Pruning pre-trained models is the classical setting of Han et al. [13] and Zhu and Gupta [30]; what our comparison adds is that gradual removal *during* adaptation then dominates one-shot pruning, and that dynamic sparse training competes only when it starts from the inherited structure. The transfer experiment (Table 4) confirms the first at 5% and 2%: gradual removal during adaptation beats one-shot pruning of the same weights, beats RigL from a random mask by seven points and keeps 5.3 and 3.5 points over pruning from scratch, at 40% of the FLOPs of a dense fine-tune; RigL from the inherited magnitude mask, which keeps the structure and rewires it, comes within 0.4 points at 28–42% of the compute, so what the inherited start buys is the structure, not the particular way it is thinned. At 0.5% the standard schedule loses to one-shot pruning and to training from scratch, but two controls trace this to time at the final density rather than to the inherited structure: ending the schedule earlier restores parity with training from scratch, and RigL from the inherited mask reaches 84.0% at a seventh of the compute. An inherited start pays off while enough of it survives and enough adaptation remains after the last cut. The developmental reading is the same: an organism inherits an over-connected cortex it did not pay for and only pays for the pruning.

Scale. Two small datasets; models from one to eleven million parameters. The ResNet-18 replication (Table 3) shows that the winner and the growth of the margin with sparsity survive a five-fold increase in model size, but ImageNet-scale data and models remain future work. The hazards we found on the CNN (layer starvation under uniform allocation, channel death under activity pruning, divergence of RigL at sub-1% mid-layer densities) suggest that the extreme-sparsity regime will need care at scale.

What the negative results constrain. A hippocampus that cannot communicate with the cortex except by distillation will transfer its bias; a token-selection rule that freezes the same background tokens in every layer will lose to random selection; an attention-sparsity mechanism is only worth its router at long sequences; a network that re-selects its connections for every input pays for the flexibility with accuracy. These are not arguments against the brain principles but against the particular reductions we tried, and they are the kind of result that is rarely published and often re-discovered.

An observation about the failures. We define a criterion and then apply it; it was formulated after the experiments, so it is an organising observation rather than a prediction. We call a reduction a *structure* when it adds a component or a decision to the network (a second network, a router, a predictor that fills gaps, a per-sample mask); a *rule* when it replaces the pruning criterion or the learning rule by a local one, activity-based for the Hebbian and drive criteria and STDP and

activity-independent for the sleep decay variants; and a *schedule* when it keeps the magnitude criterion and changes only when and how much is removed. The principle of the abstract, learning as gradual pruning, is a schedule-level change in these terms. Every structure failed. The rules were mixed: the drive criterion ends about a point behind per-layer magnitude pruning on the MLP and collapses on the CNN, the decay variants were neutral or harmful, and STDP stays at its known accuracy ceiling while sparsifying itself. Within the schedule level, the brain-derived gradual schedule beat the conventional one-shot schedule (prune-after) at every tight budget when training from scratch, which is the paper’s main result (from a pre-trained start it reverses at 0.5% under the standard schedule, Section 4); the sleep-phase magnitude pruning of Section 5 was tested only inside the sleep structure, where it was neutral. One reading, offered as a hypothesis because it rests on a single positive case, is that a substantial part of the brain’s advantage rests on compute–memory unity, the hardware property by which every synapse computes, stores and learns in place, and that a processor–memory architecture can imitate the structures only at a cost: a hippocampus that can communicate only by distillation transfers its bias, a router must spend compute to save compute, a predictor must be trained and evaluated for every token it stands in for, and a per-input subnetwork must be materialised as a per-sample copy of the weights; the activity-based rules pay a smaller version of the same cost, tracking activity in buffers separate from the weights they govern, and the decay variants add neither a module nor a buffer and simply did not help. Each failure also has a mundane explanation (bias transfer, a single shared head in class-incremental learning, attention being 5–6% of the cost, token selection carrying no information at this scale, a predicted residual carrying no more than the token it replaces once only late layers are skipped, a mask that changes faster than the weights can adapt to it), and the hippocampus–cortex structure has been made to work in software by giving the hippocampus generative capacity or a hybrid dual representation [24, 28]. We did not test the hardware reading: in our hands the gradual schedule transferred, the structures did not, and the rules were mixed.

Follow-up. The natural next question links the main result to forgetting: whether networks sparsified by prune-during interfere less across sequential tasks than dense networks of equal budget. This connects to PackNet-style continual learning [11, 18] but tests the developmental claim directly.

7 Conclusion

Pruning during learning, with a plain global magnitude criterion and a gradual schedule, is the best route to a sparse network at tight budgets on the three models we tried, ties with pruning afterwards at loose budgets, and its advantage comes mostly from the gradual schedule, not from sparsity; doing it while learning adds up to a point at the tightest budgets and costs a quarter to a third as much as pruning the trained network gradually. Starting from a pre-trained network adds 3.5–5.3 points over pruning from scratch down to about 2% of the connections without saving adaptation compute, and at 0.5% only breaks even with pruning from scratch, and only when the network is given time at the final density; dynamic sparse training from the inherited mask matches it at a fraction of the compute. The same programme’s attempts at sleep consolidation, predictive skipping, thalamic routing (pre-decided skips and pre-routed attention) and input-dependent pruning did not add to known baselines (the best of them, on Permuted MNIST, is within 0.3–0.8 points of class-balanced replay at 2.5–5× its compute), and we report them so that the next attempt can start from the right place.

Reproducibility. Code, configurations, per-run logs and the full result tables are available at <https://github.com/debtor114/subtractive-intelligence>; every number in this paper is produced by `scripts/analyze.py` and `scripts/analyze_exp12.py` from the stored run files, except the five statements taken from run logs that Appendix B lists.

References

- [1] Elahe Arani, Fahad Sarfraz, and Bahram Zonooz. Learning fast, learning slow: a general continual learning method based on complementary learning system. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=uxxFrDwrE7Y>.
- [2] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.
- [3] Aristotelis Chrysakis and Marie-Francine Moens. Online continual learning from imbalanced data. In *International Conference on Machine Learning*, 2020. URL <https://proceedings.mlr.press/v119/chrysakis20a.html>.
- [4] Peter U. Diehl and Matthew Cook. Unsupervised learning of digit recognition using spike-timing-dependent plasticity. *Frontiers in Computational Neuroscience*, 9:99, 2015. doi: 10.3389/fncom.2015.00099.
- [5] Yiting Dong, Dongcheng Zhao, Yang Li, and Yi Zeng. An unsupervised STDP-based spiking neural network inspired by biologically plausible learning rules and connections. *Neural Networks*, 165:799–808, 2023. doi: 10.1016/j.neunet.2023.06.019.
- [6] Jason K. Eshraghian, Max Ward, Emre O. Neftci, Xinxin Wang, Gregor Lenz, Girish Dwivedi, Mohammed Bennamoun, Doo Seok Jeong, and Wei D. Lu. Training spiking neural networks using lessons from deep learning. *Proceedings of the IEEE*, 111(9):1016–1054, 2023. doi: 10.1109/JPROC.2023.3308088.
- [7] Utku Evci, Trevor Gale, Jacob Menick, Pablo Samuel Castro, and Erich Elsen. Rigging the lottery: making all tickets winners. In *International Conference on Machine Learning*, 2020. URL <https://proceedings.mlr.press/v119/evci20a.html>.
- [8] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: finding sparse, trainable neural networks. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=rJl-b3RcF7>.
- [9] Karl Friston. The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11(2):127–138, 2010. doi: 10.1038/nrn2787.
- [10] Trevor Gale, Erich Elsen, and Sara Hooker. The state of sparsity in deep neural networks. *arXiv preprint arXiv:1902.09574*, 2019.
- [11] Siavash Golkar, Michael Kagan, and Kyunghyun Cho. Continual learning via neural pruning. *arXiv preprint arXiv:1903.04476*, 2019.
- [12] Ankit Gupta, Guy Dar, Shaya Goodman, David Ciprut, and Jonathan Berant. Memory-efficient transformers via top- k attention. In *Proceedings of the Second Workshop on Simple and Efficient Natural Language Processing*, pages 39–52, 2021. doi: 10.18653/v1/2021.sustainlp-1.5.

- [13] Song Han, Jeff Pool, John Tran, and William J. Dally. Learning both weights and connections for efficient neural networks. In *Advances in Neural Information Processing Systems*, 2015. URL https://proceedings.neurips.cc/paper_files/paper/2015/hash/ae0eb3eed39d2bcef4622b2499a05fe6-Abstract.html.
- [14] Torsten Hoefer, Dan Alistarh, Tal Ben-Nun, Nikoli Dryden, and Alexandra Peste. Sparsity in deep learning: pruning and growth for efficient inference and training in neural networks. *Journal of Machine Learning Research*, 22(241):1–124, 2021. URL <https://jmlr.org/papers/v22/21-0366.html>.
- [15] Peter R. Huttenlocher. Synaptic density in human frontal cortex: developmental changes and effects of aging. *Brain Research*, 163(2):195–205, 1979. doi: 10.1016/0006-8993(79)90349-4.
- [16] Dharmashan Kumaran, Demis Hassabis, and James L. McClelland. What learning systems do intelligent agents need? Complementary learning systems theory updated. *Trends in Cognitive Sciences*, 20(7):512–534, 2016. doi: 10.1016/j.tics.2016.05.004.
- [17] Zhuang Liu, Mingjie Sun, Tinghui Zhou, Gao Huang, and Trevor Darrell. Rethinking the value of network pruning. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=rJlnB3C5Ym>.
- [18] Arun Mallya and Svetlana Lazebnik. PackNet: adding multiple tasks to a single network by iterative pruning. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7765–7773, 2018. doi: 10.1109/CVPR.2018.00810.
- [19] James L. McClelland, Bruce L. McNaughton, and Randall C. O’Reilly. Why there are complementary learning systems in the hippocampus and neocortex: insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3): 419–457, 1995. doi: 10.1037/0033-295X.102.3.419.
- [20] Beren Millidge, Alexander Tschantz, and Christopher L. Buckley. Predictive coding approximates backprop along arbitrary computation graphs. *Neural Computation*, 34(6):1329–1368, 2022. doi: 10.1162/neco_a_01497.
- [21] Decebal Constantin Mocanu, Elena Mocanu, Peter Stone, Phuong H. Nguyen, Madeleine Gibescu, and Antonio Liotta. Scalable training of artificial neural networks with adaptive sparse connectivity inspired by network science. *Nature Communications*, 9:2383, 2018. doi: 10.1038/s41467-018-04316-3.
- [22] Yongming Rao, Wenliang Zhao, Benlin Liu, Jiwen Lu, Jie Zhou, and Cho-Jui Hsieh. DynamicViT: efficient vision transformers with dynamic token sparsification. In *Advances in Neural Information Processing Systems*, 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/hash/747d3443e319a22747fbb873e8b2f9f2-Abstract.html.
- [23] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9:53–68, 2021. doi: 10.1162/tacl_a_00353.
- [24] Qianqian Shi, Faqiang Liu, Hongyi Li, Guangyu Li, Luping Shi, and Rong Zhao. Hybrid neural networks for continual learning inspired by corticohippocampal circuits. *Nature Communications*, 16:1272, 2025. doi: 10.1038/s41467-025-56405-9.

- [25] Hanul Shin, Jung Kwon Lee, Jaehong Kim, and Jiwon Kim. Continual learning with deep generative replay. In *Advances in Neural Information Processing Systems*, 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/0efbe98067c6c73dba1250d2beaa81f9-Abstract.html.
- [26] Hidenori Tanaka, Daniel Kunin, Daniel L. K. Yamins, and Surya Ganguli. Pruning neural networks without any data by iteratively conserving synaptic flow. In *Advances in Neural Information Processing Systems*, 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/hash/46a4378f835dc8040c8057beb6a2da52-Abstract.html.
- [27] Giulio Tononi and Chiara Cirelli. Sleep and the price of plasticity: from synaptic and cellular homeostasis to memory consolidation and integration. *Neuron*, 81(1):12–34, 2014. doi: 10.1016/j.neuron.2013.12.025.
- [28] Gido M. van de Ven, Hava T. Siegelmann, and Andreas S. Tolias. Brain-inspired replay for continual learning with artificial neural networks. *Nature Communications*, 11:4069, 2020. doi: 10.1038/s41467-020-17866-2.
- [29] Hongxu Yin, Arash Vahdat, Jose M. Alvarez, Arun Mallya, Jan Kautz, and Pavlo Molchanov. A-ViT: adaptive tokens for efficient vision transformer. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10799–10808, 2022. doi: 10.1109/CVPR52688.2022.01054.
- [30] Michael Zhu and Suyog Gupta. To prune, or not to prune: exploring the efficacy of pruning for model compression. *arXiv preprint arXiv:1710.01878*, 2017.

A Hyper-parameters and implementation notes

Masks. Linear and convolutional layers carry a binary mask buffer; the forward pass uses $w \odot m$, masked weights are re-zeroed after every optimiser step, and pruning selects an exact top- k of the live weights by index so that tied scores (common with activity traces) cannot inflate the count.

Schedules. Prune-during: density $d(t) = d_f + (1 - d_f)(1 - t)^3$ for $t \in [0, 1]$ spanning 10–70% of training, pruning every 100 steps, with a final pruning step at 70% so that the target is met exactly. RigL/SET: drop fraction $0.3 \cdot \frac{1}{2}(1 + \cos(\pi s/S))$ every 100 steps up to $S = 75\%$ of training. Prune-after: fine-tuning for half the original epochs at lr 0.01 (CNN and ResNet-18, a fifth and a tenth of their training rates) or the original lr (MLP); the rows marked lr 0.05 in Table 2 restart the cosine schedule at the training rate instead. RigL regrowth is skipped on steps whose mixed-precision gradients are non-finite; without this guard one CIFAR-10 seed lost 93% of its connections in the first 1,000 steps. With the guard in place one CIFAR-10 seed at 1% still diverged to chance (Table 2).

Dense small sizing. MLP: hidden width h solving $784h + h^2 + 10h = B$. CNN and ResNet: all widths scaled by a common factor found by bisection on the prunable weight count. The shallow-and-wide CNN variant has two stages, each two convolutions followed by a 2×2 max-pool (base channels 128 and 256), and a third 2×2 max-pool before the classifier so that its input is 4×4 as in the six-layer network (fc 256), scaled the same way; at 1% it has channels 13 and 26 against 6, 13 and 26 for the width-scaled network.

Training. MLP: Adam, lr 10^{-3} , batch 128, one warm-up epoch then cosine, 15 epochs, no augmentation, full training and test sets resident on the GPU. CNN: SGD with Nesterov momentum 0.9, lr 0.05, weight decay 5×10^{-4} on weights only, batch 128, mixed precision, 20 epochs, random crop (padding 4) and horizontal flip performed on the GPU. ResNet-18: the same with lr 0.1.

Spiking network. 784 Poisson inputs at 32 Hz per unit intensity, 250 ms per image, 1 ms steps; 400 LIF neurons with adaptive thresholds ($\theta^+ = 0.05$), 5 ms refractory period, recurrent inhibition of 17.5 mV; trace STDP with $\eta_{\text{pre}} = 10^{-4}$, $\eta_{\text{post}} = 10^{-2}$; input weights normalised to a column sum of 78.4; batches of 32 images with summed updates. Labels are assigned to neurons by their class-wise mean response on 10k training images.

B Full result tables

Tables 7–22 give every number behind the figures and tables of the main text, as mean $\pm$ standard deviation over seeds. FLOPs are analytic counts for one sample (inference) or accumulated over training at the actual density of each step. Five statements in the text rest on run logs rather than on these tables: the two-epoch collapse of the unnormalised drive rule on the CNN, the accuracy trajectory of prune-after on CIFAR-10 at its pruning step, the lowest point of the prune-during trajectory in Figure 1, the initial weight statistics of the spiking network, and its final Gini coefficient.

density	arm	seeds	active weights	test acc	ECE	infer FLOPs	cum. train FLOPs	samples to 97%
1	dense big (no pruning)	3	1,861,632	0.9871 $\pm$ 0.0003	0.009 $\pm$ 0.000	3.72×10^5	1.01×10^{13}	87336 $\pm$ 7781
0.1	dense small (additive)	3	185,787	0.9836 $\pm$ 0.0003	0.009 $\pm$ 0.000	3.72×10^5	1.00×10^{12}	133096 $\pm$ 6988
0.1	prune-after: one-shot magnitude (global) + fine-tune	3	186,163	0.9869 $\pm$ 0.0003	0.008 $\pm$ 0.000	3.72×10^5	1.06×10^{13}	87336 $\pm$ 7781
0.1	prune-after: gradual magnitude (global) during fine-tune	3	186,163	0.9870 $\pm$ 0.0003	0.006 $\pm$ 0.000	3.72×10^5	1.18×10^{13}	87336 $\pm$ 7781
0.1	prune-during: magnitude (uniform per layer)	3	186,164	0.9867 $\pm$ 0.0003	0.007 $\pm$ 0.000	3.72×10^5	3.33×10^{12}	87336 $\pm$ 7781
0.1	prune-during: magnitude (global)	3	186,163	0.9867 $\pm$ 0.0002	0.008 $\pm$ 0.000	3.72×10^5	3.33×10^{12}	87336 $\pm$ 7781
0.1	prune-during: activity, Hebbian (uniform per layer)	3	186,164	0.9819 $\pm$ 0.0007	0.004 $\pm$ 0.000	3.72×10^5	3.33×10^{12}	87336 $\pm$ 7781
0.1	prune-during: abs(w) $\times$ activity, Hebbian (uniform per layer)	3	186,164	0.9857 $\pm$ 0.0004	0.006 $\pm$ 0.000	3.72×10^5	3.33×10^{12}	87336 $\pm$ 7781
0.1	prune-during: synaptic drive (uniform per layer)	3	186,164	0.9853 $\pm$ 0.0004	0.007 $\pm$ 0.000	3.72×10^5	3.33×10^{12}	87336 $\pm$ 7781
0.1	prune-during: random (uniform per layer)	3	186,164	0.9823 $\pm$ 0.0012	0.004 $\pm$ 0.000	3.72×10^5	3.33×10^{12}	87336 $\pm$ 7781
0.1	SET (dynamic sparse training, random regrowth)	3	186,164	0.9838 $\pm$ 0.0004	0.010 $\pm$ 0.001	3.72×10^5	1.01×10^{12}	138766 $\pm$ 2328
0.1	RigL (dynamic sparse training, gradient regrowth)	3	186,164	0.9853 $\pm$ 0.0011	0.008 $\pm$ 0.001	3.72×10^5	1.01×10^{12}	123519 $\pm$ 1928
0.1	static random sparse	3	186,164	0.9818 $\pm$ 0.0002	0.006 $\pm$ 0.000	3.72×10^5	1.01×10^{12}	206096 $\pm$ 4555
0.05	dense small (additive)	3	93,392	0.9801 $\pm$ 0.0003	0.009 $\pm$ 0.001	1.87×10^5	5.04×10^{11}	186445 $\pm$ 12401
0.05	prune-after: one-shot magnitude (global) + fine-tune	3	93,082	0.9873 $\pm$ 0.0003	0.006 $\pm$ 0.000	1.86×10^5	1.03×10^{13}	87336 $\pm$ 7781
0.05	prune-after: gradual magnitude (global) during fine-tune	3	93,082	0.9864 $\pm$ 0.0002	0.004 $\pm$ 0.000	1.86×10^5	1.16×10^{13}	87336 $\pm$ 7781
0.05	prune-during: magnitude (uniform per layer)	3	93,082	0.9868 $\pm$ 0.0006	0.006 $\pm$ 0.001	1.86×10^5	2.96×10^{12}	87336 $\pm$ 7781
0.05	prune-during: magnitude (global)	3	93,082	0.9872 $\pm$ 0.0003	0.006 $\pm$ 0.001	1.86×10^5	2.96×10^{12}	87336 $\pm$ 7781
0.05	prune-during: activity, Hebbian (uniform per layer)	3	93,082	0.9787 $\pm$ 0.0001	0.005 $\pm$ 0.001	1.86×10^5	2.96×10^{12}	87336 $\pm$ 7781
0.05	prune-during: abs(w) $\times$ activity, Hebbian (uniform per layer)	3	93,082	0.9842 $\pm$ 0.0004	0.004 $\pm$ 0.000	1.86×10^5	2.96×10^{12}	87336 $\pm$ 7781
0.05	prune-during: synaptic drive (uniform per layer)	3	93,082	0.9850 $\pm$ 0.0003	0.005 $\pm$ 0.000	1.86×10^5	2.96×10^{12}	87336 $\pm$ 7781
0.05	prune-during: random (uniform per layer)	3	93,082	0.9713 $\pm$ 0.0011	0.015 $\pm$ 0.000	1.86×10^5	2.96×10^{12}	87336 $\pm$ 7781
0.05	SET (dynamic sparse training, random regrowth)	3	93,082	0.9819 $\pm$ 0.0009	0.006 $\pm$ 0.001	1.86×10^5	5.03×10^{11}	209140 $\pm$ 5774
0.05	RigL (dynamic sparse training, gradient regrowth)	3	93,082	0.9822 $\pm$ 0.0004	0.008 $\pm$ 0.000	1.86×10^5	5.03×10^{11}	153687 $\pm$ 5905
0.05	static random sparse	3	93,082	0.9754 $\pm$ 0.0003	0.004 $\pm$ 0.001	1.86×10^5	5.03×10^{11}	378278 $\pm$ 18120
0.02	dense small (additive)	3	36,872	0.9730 $\pm$ 0.0010	0.006 $\pm$ 0.000	7.37×10^4	1.99×10^{11}	436301 $\pm$ 46700
0.02	prune-after: one-shot magnitude (global) + fine-tune	3	37,233	0.9838 $\pm$ 0.0008	0.003 $\pm$ 0.001	7.45×10^4	1.02×10^{13}	87336 $\pm$ 7781
0.02	prune-after: gradual magnitude (global) during fine-tune	3	37,233	0.9849 $\pm$ 0.0004	0.004 $\pm$ 0.000	7.45×10^4	1.15×10^{13}	87336 $\pm$ 7781
0.02	prune-during: magnitude (uniform per layer)	3	37,233	0.9855 $\pm$ 0.0007	0.003 $\pm$ 0.000	7.45×10^4	2.73×10^{12}	87336 $\pm$ 7781
0.02	prune-during: magnitude (global)	3	37,233	0.9865 $\pm$ 0.0002	0.003 $\pm$ 0.001	7.45×10^4	2.73×10^{12}	87336 $\pm$ 7781
0.02	prune-during: activity, Hebbian (uniform per layer)	3	37,233	0.9655 $\pm$ 0.0009	0.020 $\pm$ 0.001	7.45×10^4	2.73×10^{12}	87336 $\pm$ 7781
0.02	prune-during: abs(w) $\times$ activity, Hebbian (uniform per layer)	3	37,233	0.9805 $\pm$ 0.0004	0.005 $\pm$ 0.001	7.45×10^4	2.73×10^{12}	87336 $\pm$ 7781
0.02	prune-during: synaptic drive (uniform per layer)	3	37,233	0.9822 $\pm$ 0.0003	0.004 $\pm$ 0.000	7.45×10^4	2.73×10^{12}	87336 $\pm$ 7781
0.02	prune-during: random (uniform per layer)	3	37,233	0.9339 $\pm$ 0.0015	0.034 $\pm$ 0.002	7.45×10^4	2.73×10^{12}	87336 $\pm$ 7781
0.02	SET (dynamic sparse training, random regrowth)	3	37,233	0.9743 $\pm$ 0.0006	0.005 $\pm$ 0.000	7.45×10^4	2.01×10^{11}	467683 $\pm$ 20709
0.02	RigL (dynamic sparse training, gradient regrowth)	3	37,233	0.9785 $\pm$ 0.0004	0.005 $\pm$ 0.001	7.45×10^4	2.01×10^{11}	245749 $\pm$ 12815
0.02	static random sparse	3	37,233	0.9590 $\pm$ 0.0013	0.009 $\pm$ 0.001	7.45×10^4	2.01×10^{11}	-
0.01	dense small (additive)	3	18,791	0.9630 $\pm$ 0.0012	0.004 $\pm$ 0.001	3.76×10^4	1.01×10^{11}	-
0.01	prune-after: one-shot magnitude (global) + fine-tune	3	18,616	0.9752 $\pm$ 0.0020	0.006 $\pm$ 0.000	3.72×10^4	1.01×10^{13}	87336 $\pm$ 7781
0.01	prune-after: gradual magnitude (global) during fine-tune	3	18,616	0.9794 $\pm$ 0.0006	0.009 $\pm$ 0.000	3.72×10^4	1.14×10^{13}	87336 $\pm$ 7781
0.01	prune-during: magnitude (uniform per layer)	3	18,616	0.9823 $\pm$ 0.0001	0.007 $\pm$ 0.000	3.72×10^4	2.66×10^{12}	87336 $\pm$ 7781
0.01	prune-during: magnitude (global)	3	18,616	0.9844 $\pm$ 0.0003	0.004 $\pm$ 0.000	3.72×10^4	2.66×10^{12}	87336 $\pm$ 7781
0.01	prune-during: activity, Hebbian (uniform per layer)	3	18,616	0.9366 $\pm$ 0.0015	0.037 $\pm$ 0.001	3.72×10^4	2.66×10^{12}	87336 $\pm$ 7781
0.01	prune-during: abs(w) $\times$ activity, Hebbian (uniform per layer)	3	18,616	0.9725 $\pm$ 0.0008	0.012 $\pm$ 0.001	3.72×10^4	2.66×10^{12}	87336 $\pm$ 7781
0.01	prune-during: synaptic drive (uniform per layer)	3	18,616	0.9756 $\pm$ 0.0010	0.012 $\pm$ 0.000	3.72×10^4	2.66×10^{12}	87336 $\pm$ 7781
0.01	prune-during: random (uniform per layer)	3	18,616	0.8913 $\pm$ 0.0060	0.071 $\pm$ 0.003	3.72×10^4	2.66×10^{12}	87336 $\pm$ 7781
0.01	SET (dynamic sparse training, random regrowth)	3	18,616	0.9603 $\pm$ 0.0010	0.008 $\pm$ 0.001	3.72×10^4	1.01×10^{11}	-
0.01	RigL (dynamic sparse training, gradient regrowth)	3	18,616	0.9741 $\pm$ 0.0016	0.004 $\pm$ 0.001	3.72×10^4	1.01×10^{11}	444658 $\pm$ 67026
0.01	RigL, 3x training length	3	18,616	0.9772 $\pm$ 0.0008	0.013 $\pm$ 0.001	3.72×10^4	3.02×10^{11}	445880 $\pm$ 52617
0.01	static random sparse	3	18,616	0.9259 $\pm$ 0.0034	0.015 $\pm$ 0.001	3.72×10^4	1.01×10^{11}	-
0.005	dense small (additive)	3	9,672	0.9407 $\pm$ 0.0010	0.009 $\pm$ 0.001	1.93×10^4	5.22×10^{10}	-
0.005	prune-after: one-shot magnitude (global) + fine-tune	3	9,308	0.9451 $\pm$ 0.0060	0.017 $\pm$ 0.006	1.86×10^4	1.01×10^{13}	87336 $\pm$ 7781
0.005	prune-after: gradual magnitude (global) during fine-tune	3	9,308	0.9692 $\pm$ 0.0012	0.020 $\pm$ 0.001	1.86×10^4	1.14×10^{13}	87336 $\pm$ 7781
0.005	prune-during: magnitude (uniform per layer)	3	9,308	0.9704 $\pm$ 0.0005	0.015 $\pm$ 0.001	1.86×10^4	2.62×10^{12}	87336 $\pm$ 7781
0.005	prune-during: magnitude (global)	3	9,308	0.9781 $\pm$ 0.0002	0.006 $\pm$ 0.000	1.86×10^4	2.62×10^{12}	87336 $\pm$ 7781
0.005	prune-during: activity, Hebbian (uniform per layer)	3	9,322	0.8885 $\pm$ 0.0109	0.100 $\pm$ 0.037	1.86×10^4	2.62×10^{12}	87336 $\pm$ 7781
0.005	prune-during: abs(w) $\times$ activity, Hebbian (uniform per layer)	3	9,308	0.9522 $\pm$ 0.0013	0.038 $\pm$ 0.016	1.86×10^4	2.62×10^{12}	87336 $\pm$ 7781
0.005	prune-during: synaptic drive (uniform per layer)	3	9,308	0.9603 $\pm$ 0.0020	0.025 $\pm$ 0.000	1.86×10^4	2.62×10^{12}	87336 $\pm$ 7781
0.005	prune-during: random (uniform per layer)	3	9,308	0.7184 $\pm$ 0.0230	0.157 $\pm$ 0.017	1.86×10^4	2.62×10^{12}	87336 $\pm$ 7781
0.005	SET (dynamic sparse training, random regrowth)	3	9,308	0.9365 $\pm$ 0.0011	0.012 $\pm$ 0.001	1.86×10^4	5.03×10^{10}	-
0.005	RigL (dynamic sparse training, gradient regrowth)	3	9,308	0.9631 $\pm$ 0.0027	0.005 $\pm$ 0.001	1.86×10^4	5.03×10^{10}	-
0.005	RigL, 3x training length	3	9,308	0.9746 $\pm$ 0.0010	0.008 $\pm$ 0.001	1.86×10^4	1.51×10^{11}	844057 $\pm$ 92020
0.005	static random sparse	3	9,308	0.7271 $\pm$ 0.0271	0.053 $\pm$ 0.012	1.86×10^4	5.03×10^{10}	-

Table 7: MNIST MLP (784-1024-1024-10, 1.86M weights), all arms and budgets, 3 seeds. Density is the final active fraction of the over-parameterised network; dense small is a width-matched network at the same budget. Training FLOPs count three forward passes per step at the actual density of that step.

density	arm	seeds	active weights	test acc	ECE	infer FLOPs	cum. train FLOPs	samples to 85%
1	dense big (no pruning)	3	2,195,648	0.9133 $\pm$ 0.0010	0.031 $\pm$ 0.001	3.08×10^8	9.23×10^{14}	411367 $\pm$ 15611
0.1	dense small (additive)	3	219,735	0.8729 $\pm$ 0.0007	0.029 $\pm$ 0.001	3.10×10^7	9.31×10^{13}	632089 $\pm$ 19636
0.1	dense small, shallow and wide (additive)	3	219,918	0.8685 $\pm$ 0.0026	0.025 $\pm$ 0.002	8.09×10^7	2.43×10^{14}	676291 $\pm$ 26535
0.1	prune-after: one-shot magnitude (global) + fine-tune	3	219,565	0.9072 $\pm$ 0.0014	0.036 $\pm$ 0.001	8.19×10^7	1.05×10^{15}	411367 $\pm$ 15611
0.1	prune-after: gradual magnitude (global) during fine-tune	3	219,565	0.9051 $\pm$ 0.0023	0.036 $\pm$ 0.002	8.05×10^7	1.15×10^{15}	440414 $\pm$ 23340
0.1	prune-during: magnitude (global)	3	219,565	0.9028 $\pm$ 0.0029	0.033 $\pm$ 0.001	7.37×10^7	4.19×10^{14}	387708 $\pm$ 26819
0.1	prune-during: magnitude (ERK per layer)	3	219,565	0.8930 $\pm$ 0.0004	0.030 $\pm$ 0.001	2.69×10^7	2.80×10^{14}	374564 $\pm$ 22672
0.1	prune-during: synaptic drive, per-neuron normalised (ERK per layer)	3	219,565	0.8759 $\pm$ 0.0033	0.026 $\pm$ 0.003	2.69×10^7	2.80×10^{14}	633096 $\pm$ 45722
0.1	RigL (dynamic sparse training, gradient regrowth)	3	219,565	0.8759 $\pm$ 0.0005	0.026 $\pm$ 0.002	2.69×10^7	8.07×10^{13}	625325 $\pm$ 28326
0.03	dense small (additive)	3	67,905	0.8284 $\pm$ 0.0010	0.024 $\pm$ 0.001	9.72×10^6	2.92×10^{13}	-
0.03	dense small, shallow and wide (additive)	3	68,264	0.8391 $\pm$ 0.0008	0.019 $\pm$ 0.001	2.57×10^7	7.72×10^{13}	-
0.03	prune-after: one-shot magnitude (global) + fine-tune	3	65,869	0.8838 $\pm$ 0.0022	0.026 $\pm$ 0.001	3.58×10^7	9.77×10^{14}	411367 $\pm$ 15611
0.03	prune-after: gradual magnitude (global) during fine-tune	3	65,869	0.8797 $\pm$ 0.0011	0.022 $\pm$ 0.001	3.42×10^7	1.11×10^{15}	440414 $\pm$ 23340
0.03	prune-after: one-shot magnitude (global) + fine-tune at lr 0.05	3	65,869	0.8837 $\pm$ 0.0012	0.031 $\pm$ 0.002	3.59×10^7	9.77×10^{14}	410359 $\pm$ 20729
0.03	prune-after: gradual magnitude (global) during fine-tune at lr 0.05	3	65,869	0.8922 $\pm$ 0.0015	0.028 $\pm$ 0.003	3.09×10^7	1.11×10^{15}	404766 $\pm$ 38638
0.03	prune-during: magnitude (global)	3	65,869	0.8908 $\pm$ 0.0017	0.028 $\pm$ 0.002	2.98×10^7	3.40×10^{14}	391024 $\pm$ 6160
0.03	prune-during: magnitude (ERK per layer)	3	65,870	0.8692 $\pm$ 0.0026	0.023 $\pm$ 0.003	8.38×10^6	2.39×10^{14}	495322 $\pm$ 145564
0.03	prune-during: synaptic drive, per-neuron normalised (ERK per layer)	3	65,870	0.7928 $\pm$ 0.0048	0.020 $\pm$ 0.003	8.38×10^6	2.39×10^{14}	-
0.03	RigL (dynamic sparse training, gradient regrowth)	3	65,870	0.8353 $\pm$ 0.0019	0.018 $\pm$ 0.001	8.38×10^6	2.52×10^{13}	-
0.03	RigL, 3x training length	3	65,870	0.8702 $\pm$ 0.0022	0.025 $\pm$ 0.001	8.38×10^6	7.55×10^{13}	1909949 $\pm$ 39835
0.01	dense small (additive)	3	22,911	0.7709 $\pm$ 0.0036	0.020 $\pm$ 0.004	3.32×10^6	9.97×10^{12}	-
0.01	dense small, shallow and wide (additive)	3	22,074	0.7836 $\pm$ 0.0026	0.016 $\pm$ 0.002	8.53×10^6	2.56×10^{13}	-
0.01	prune-after: one-shot magnitude (global) + fine-tune	3	21,956	0.8051 $\pm$ 0.0010	0.015 $\pm$ 0.003	1.60×10^7	9.47×10^{14}	411367 $\pm$ 15611
0.01	prune-after: gradual magnitude (global) during fine-tune	3	21,956	0.8192 $\pm$ 0.0021	0.012 $\pm$ 0.002	1.47×10^7	1.09×10^{15}	440414 $\pm$ 23340
0.01	prune-after: one-shot magnitude (global) + fine-tune at lr 0.05	3	21,956	0.8113 $\pm$ 0.0026	0.020 $\pm$ 0.001	1.60×10^7	9.47×10^{14}	410804 $\pm$ 17276
0.01	prune-after: gradual magnitude (global) during fine-tune at lr 0.05	3	21,956	0.8481 $\pm$ 0.0018	0.021 $\pm$ 0.000	1.27×10^7	1.09×10^{15}	419317 $\pm$ 25110
0.01	prune-during: magnitude (global)	3	21,956	0.8570 $\pm$ 0.0020	0.023 $\pm$ 0.001	1.16×10^7	3.09×10^{14}	390029 $\pm$ 6850
0.01	prune-during: magnitude (ERK per layer)	3	21,957	0.8134 $\pm$ 0.0007	0.015 $\pm$ 0.001	2.78×10^6	2.27×10^{14}	-
0.01	prune-during: synaptic drive, per-neuron normalised (ERK per layer)	3	21,957	0.6004 $\pm$ 0.0141	0.013 $\pm$ 0.006	2.78×10^6	2.27×10^{14}	-
0.01	RigL (dynamic sparse training, gradient regrowth)	3	21,957	0.5392 $\pm$ 0.3110	0.008 $\pm$ 0.006	2.78×10^6	8.35×10^{12}	-
0.01	RigL, 3x training length	3	21,957	0.5666 $\pm$ 0.3299	0.010 $\pm$ 0.007	2.78×10^6	2.50×10^{13}	-

Table 8: CIFAR-10 CNN (2.2M weights), all arms and budgets, 3 seeds. Columns as in Table 7; the data-efficiency column is the number of training samples until 85% test accuracy.

density	arm	seeds	active weights	test acc	ECE	infer FLOPs	cum. train FLOPs	samples to 85%
1	dense big (no pruning)	3	11,164,352	0.9248 $\pm$ 0.0026	0.035 $\pm$ 0.001	1.11×10^9	3.33×10^{15}	436855 $\pm$ 29455
0.2	dense small (additive)	2	2,239,484	0.9093 $\pm$ 0.0000	0.035 $\pm$ 0.002	2.25×10^8	6.74×10^{14}	514366 $\pm$ 32163
0.2	prune-during: magnitude (uniform per layer)	2	2,232,871	0.9179 $\pm$ 0.0009	0.038 $\pm$ 0.000	2.22×10^8	1.35×10^{15}	382401 $\pm$ 5235
0.2	prune-during: magnitude (global)	2	2,232,870	0.9189 $\pm$ 0.0013	0.038 $\pm$ 0.000	3.45×10^8	1.65×10^{15}	377044 $\pm$ 750
0.05	dense small (additive)	3	562,229	0.8866 $\pm$ 0.0018	0.030 $\pm$ 0.002	5.58×10^7	1.67×10^{14}	665417 $\pm$ 15510
0.05	prune-after: one-shot magnitude (global) + fine-tune	3	558,218	0.9227 $\pm$ 0.0006	0.035 $\pm$ 0.001	2.71×10^8	3.74×10^{15}	434143 $\pm$ 30338
0.05	prune-during: magnitude (uniform per layer)	2	558,217	0.9108 $\pm$ 0.0002	0.037 $\pm$ 0.001	5.55×10^7	9.79×10^{14}	387334 $\pm$ 8668
0.05	prune-during: magnitude (global)	3	558,218	0.9196 $\pm$ 0.0023	0.034 $\pm$ 0.001	1.85×10^8	1.33×10^{15}	384521 $\pm$ 15853
0.05	prune-during: magnitude (ERK per layer)	3	558,213	0.9203 $\pm$ 0.0009	0.034 $\pm$ 0.001	1.39×10^8	1.43×10^{15}	399622 $\pm$ 18599
0.05	RigL (dynamic sparse training, gradient regrowth)	3	558,213	0.9097 $\pm$ 0.0016	0.034 $\pm$ 0.001	1.39×10^8	4.17×10^{14}	524129 $\pm$ 20642
0.02	dense small (additive)	3	225,892	0.8647 $\pm$ 0.0030	0.026 $\pm$ 0.000	2.25×10^7	6.76×10^{13}	801128 $\pm$ 18275
0.02	prune-after: one-shot magnitude (global) + fine-tune	3	223,287	0.9094 $\pm$ 0.0018	0.027 $\pm$ 0.002	1.49×10^8	3.56×10^{15}	434143 $\pm$ 30338
0.02	prune-during: magnitude (global)	3	223,287	0.9146 $\pm$ 0.0015	0.034 $\pm$ 0.002	1.07×10^8	1.20×10^{15}	388422 $\pm$ 24257
0.02	prune-during: magnitude (ERK per layer)	3	223,288	0.9125 $\pm$ 0.0012	0.032 $\pm$ 0.001	5.71×10^7	1.26×10^{15}	394149 $\pm$ 11727
0.02	RigL (dynamic sparse training, gradient regrowth)	3	223,288	0.8950 $\pm$ 0.0018	0.030 $\pm$ 0.003	5.71×10^7	1.71×10^{14}	597590 $\pm$ 17418
0.005	dense small (additive)	3	55,872	0.8131 $\pm$ 0.0026	0.021 $\pm$ 0.001	6.12×10^6	1.84×10^{13}	-
0.005	prune-after: one-shot magnitude (global) + fine-tune	3	55,822	0.8364 $\pm$ 0.0015	0.016 $\pm$ 0.001	5.27×10^7	3.41×10^{15}	445811 $\pm$ 28637
0.005	prune-during: magnitude (uniform per layer)	2	55,819	0.8311 $\pm$ 0.0006	0.022 $\pm$ 0.001	5.55×10^6	8.67×10^{14}	392455 $\pm$ 2533
0.005	prune-during: magnitude (global)	3	55,822	0.8896 $\pm$ 0.0003	0.027 $\pm$ 0.001	3.40×10^7	1.09×10^{15}	401879 $\pm$ 17074
0.005	prune-during: magnitude (ERK per layer)	3	55,820	0.8783 $\pm$ 0.0022	0.024 $\pm$ 0.001	1.38×10^7	1.18×10^{15}	391071 $\pm$ 8159
0.005	RigL (dynamic sparse training, gradient regrowth)	3	55,820	0.8486 $\pm$ 0.0030	0.026 $\pm$ 0.001	1.38×10^7	4.15×10^{13}	907936

Table 9: CIFAR-10 ResNet-18 (11.2M weights), all arms and budgets. Columns as in Table 7; samples to 85% test accuracy.

density	arm	seeds	active conv weights	init acc	final acc	best acc	ECE	infer FLOPs	adaptation train FLOPs	samples to 90%
1	pre-trained, dense fine-tune (no pruning)	2	11,166,912	0.1085 ± 0.0170	0.9593 ± 0.0000	0.9598 ± 0.0000	0.018 ± 0.001	1.18 × 10 ⁹	1.78 × 10 ⁵	52515 ± 1566
0.05	from scratch, dense small (width-scaled)	2	562,759	0.1000 ± 0.0000	0.8484 ± 0.0004	0.8486 ± 0.0006	0.022 ± 0.001	7.19 × 10 ⁷	1.08 × 10 ⁻⁴	-
0.05	from scratch, prune-during (global magnitude)	2	558,346	0.1000 ± 0.0000	0.8835 ± 0.0007	0.8838 ± 0.0010	0.031 ± 0.001	2.40 × 10 ⁸	7.72 × 10 ⁻⁴	-
0.05	pre-trained, one-shot prune then fine-tune	2	558,346	0.1000 ± 0.0000	0.9306 ± 0.0012	0.9316 ± 0.0007	0.020 ± 0.001	2.01 × 10 ⁸	3.02 × 10 ⁻⁴	165526 ± 4694
0.05	pre-trained, random ERK mask + RigL regrow	2	558,347	0.1000 ± 0.0000	0.8690 ± 0.0021	0.8692 ± 0.0020	0.023 ± 0.001	1.78 × 10 ⁸	2.68 × 10 ⁻⁴	-
0.05	pre-trained, magnitude mask + RigL regrow (keeps inherited structure)	2	558,346	0.1000 ± 0.0000	0.9329 ± 0.0012	0.9336 ± 0.0015	0.018 ± 0.002	2.01 × 10 ⁸	3.02 × 10 ⁻⁴	181334 ± 2491
0.05	pre-trained, prune-during adaptation (ERK layer budget)	2	558,347	0.1085 ± 0.0170	0.9343 ± 0.0005	0.9356 ± 0.0003	0.015 ± 0.001	1.78 × 10 ⁸	8.10 × 10 ⁻⁴	52380 ± 1430
0.05	pre-trained, prune-during adaptation (global magnitude)	2	558,346	0.1085 ± 0.0170	0.9365 ± 0.0008	0.9377 ± 0.0005	0.017 ± 0.001	2.03 × 10 ⁸	7.26 × 10 ⁻⁴	52276 ± 1327
0.02	from scratch, dense small (width-scaled)	2	225,242	0.0991 ± 0.0048	0.8240 ± 0.0006	0.8241 ± 0.0005	0.022 ± 0.001	3.29 × 10 ⁷	4.93 × 10 ⁻³	-
0.02	from scratch, prune-during (global magnitude)	2	223,338	0.1000 ± 0.0000	0.8747 ± 0.0016	0.8752 ± 0.0011	0.031 ± 0.001	1.44 × 10 ⁸	6.94 × 10 ⁻⁴	-
0.02	pre-trained, one-shot prune then fine-tune	2	223,338	0.1000 ± 0.0000	0.9029 ± 0.0010	0.9038 ± 0.0000	0.023 ± 0.000	1.23 × 10 ⁸	1.85 × 10 ⁻⁴	401984 ± 12502
0.02	pre-trained, random ERK mask + RigL regrow	2	223,341	0.1000 ± 0.0000	0.8355 ± 0.0020	0.8355 ± 0.0019	0.025 ± 0.000	7.17 × 10 ⁷	1.08 × 10 ⁻⁴	-
0.02	pre-trained, magnitude mask + RigL regrow (keeps inherited structure)	2	223,338	0.1000 ± 0.0000	0.9067 ± 0.0006	0.9073 ± 0.0010	0.025 ± 0.002	1.23 × 10 ⁸	1.85 × 10 ⁻⁴	358505 ± 839
0.02	pre-trained, prune-during adaptation (ERK layer budget)	2	223,341	0.1085 ± 0.0170	0.8966 ± 0.0019	0.9292 ± 0.0008	0.020 ± 0.000	7.17 × 10 ⁷	7.10 × 10 ⁻⁴	52567 ± 1617
0.02	pre-trained, prune-during adaptation (global magnitude)	2	223,338	0.1085 ± 0.0170	0.9068 ± 0.0008	0.9329 ± 0.0010	0.019 ± 0.001	1.25 × 10 ⁸	6.57 × 10 ⁻⁴	52360 ± 1410
0.005	from scratch, dense small (width-scaled)	2	56,112	0.1000 ± 0.0000	0.7654 ± 0.0001	0.7664 ± 0.0011	0.022 ± 0.001	1.19 × 10 ⁷	1.78 × 10 ⁻³	-
0.005	from scratch, prune-during (global magnitude)	2	55,835	0.1000 ± 0.0000	0.8350 ± 0.0037	0.8384 ± 0.0034	0.028 ± 0.002	5.39 × 10 ⁷	6.27 × 10 ⁻⁴	-
0.005	pre-trained, one-shot prune then fine-tune	2	55,835	0.0884 ± 0.0116	0.8171 ± 0.0014	0.8184 ± 0.0019	0.019 ± 0.002	5.64 × 10 ⁷	8.45 × 10 ⁻³	-
0.005	pre-trained, random ERK mask + RigL regrow	2	55,834	0.1000 ± 0.0000	0.7682 ± 0.0037	0.7690 ± 0.0044	0.023 ± 0.002	1.79 × 10 ⁷	2.69 × 10 ⁻³	-
0.005	pre-trained, magnitude mask + RigL regrow (keeps inherited structure)	2	55,835	0.0882 ± 0.0118	0.8400 ± 0.0025	0.8416 ± 0.0030	0.024 ± 0.002	5.64 × 10 ⁷	8.45 × 10 ⁻³	-
0.005	pre-trained, prune-during adaptation (ERK layer budget)	2	55,834	0.1085 ± 0.0170	0.7883 ± 0.0022	0.9314 ± 0.0006	0.020 ± 0.000	1.79 × 10 ⁷	6.59 × 10 ⁻⁴	52422 ± 1473
0.005	pre-trained, prune-during adaptation (global magnitude)	2	55,835	0.1085 ± 0.0170	0.8087 ± 0.0012	0.9306 ± 0.0002	0.016 ± 0.000	5.68 × 10 ⁷	6.04 × 10 ⁻⁴	52422 ± 1473
0.005	pre-trained, prune-during adaptation, pruning ends at 50% of adaptation (global magnitude)	2	55,835	0.1084 ± 0.0168	0.8365 ± 0.0013	0.9222 ± 0.0013	0.019 ± 0.002	5.64 × 10 ⁷	4.95 × 10 ⁻⁴	43093 ± 5242

Table 10: ImageNet-pre-trained ResNet-18 adapted to CIFAR-10 (128×128 input, 10 epochs) while being pruned. Density is relative to the 11.17M convolutional weights; the classifier is kept dense in every arm. Init acc is the accuracy before adaptation (fresh classifier). Adaptation FLOPs exclude the ImageNet pre-training, which is common to all pre-trained arms.

method	seeds	avg acc (final)	forgetting	retention	ECE (all classes)	train FLOPs	active params (predictor)	unknown-class AUROC (softmax)	unknown-class AUROC (fast/slow agreement)
fine-tune (no protection)	3	0.1969 ± 0.0004	0.9950 ± 0.0001	0.000 ± 0.000	0.731 ± 0.005	2.90×10^{11}	269,322	0.649 ± 0.022	-
experience replay (buffer 500)	3	0.8515 ± 0.0043	0.1747 ± 0.0062	0.824 ± 0.006	0.087 ± 0.006	5.82×10^{11}	269,322	0.845 ± 0.007	-
ER, class-balanced batches (buffer 500)	3	0.8431 ± 0.0104	0.1880 ± 0.0132	0.811 ± 0.013	0.105 ± 0.008	1.16×10^{12}	269,322	0.842 ± 0.021	-
CLS + sleep (replay + distill)	3	0.8784 ± 0.0068	0.1302 ± 0.0087	0.868 ± 0.009	0.069 ± 0.005	2.87×10^{11}	269,322	0.855 ± 0.017	-
CLS + sleep (no distill)	3	0.7871 ± 0.0164	0.2542 ± 0.0213	0.744 ± 0.021	0.123 ± 0.028	1.84×10^{12}	269,322	0.774 ± 0.015	0.511 ± 0.012
CLS + sleep + downscale 0.1	3	0.8497 ± 0.0059	0.1791 ± 0.0069	0.820 ± 0.007	0.090 ± 0.003	1.84×10^{12}	269,322	0.834 ± 0.004	0.508 ± 0.002
CLS + sleep + prune 5%/sleep (magnitude)	3	0.7744 ± 0.0211	0.2700 ± 0.0273	0.728 ± 0.028	0.133 ± 0.027	1.84×10^{12}	269,322	0.776 ± 0.013	0.512 ± 0.012
CLS + sleep + prune 5%/sleep (activity)	3	0.7920 ± 0.0113	0.2485 ± 0.0158	0.750 ± 0.016	0.118 ± 0.025	1.84×10^{12}	208,514	0.776 ± 0.017	0.511 ± 0.012
CLS + sleep + downscale + prune	3	0.7803 ± 0.0156	0.2582 ± 0.0191	0.739 ± 0.020	0.107 ± 0.022	1.84×10^{12}	208,514	0.744 ± 0.016	0.521 ± 0.012
CLS2: balanced sleep replay; no distill, dense fast	3	0.7875 ± 0.0198	0.2540 ± 0.0247	0.744 ± 0.025	0.121 ± 0.024	1.84×10^{12}	208,514	0.771 ± 0.022	0.510 ± 0.013
CLS2: balanced sleep replay; no distill, dense fast [predict: fast]	3	0.1969 ± 0.0002	0.9964 ± 0.0004	0.000 ± 0.000	-	-	-	-	-
CLS2: balanced sleep replay; no distill, dense fast [predict: max_conf]	3	0.6794 ± 0.0256	0.3947 ± 0.0325	0.694 ± 0.032	0.080 ± 0.005	3.27×10^{12}	269,322	0.857 ± 0.017	0.505 ± 0.001
CLS2 + sparse fast (k-WTA 10%) [predict: fast]	3	0.8793 ± 0.0059	0.1528 ± 0.0075	0.846 ± 0.008	-	-	-	-	-
CLS2 + sparse fast (k-WTA 10%) [predict: max_conf]	3	0.1969 ± 0.0002	0.9973 ± 0.0004	0.000 ± 0.000	-	-	-	-	-
CLS2 + sparse fast (k-WTA 10%) [predict: max_conf]	3	0.6820 ± 0.0116	0.3924 ± 0.0144	0.607 ± 0.015	0.080 ± 0.005	3.27×10^{12}	269,322	0.857 ± 0.017	0.511 ± 0.002
CLS2 + sparse fast (k-WTA 5%) [predict: fast]	3	0.1969 ± 0.0006	0.9965 ± 0.0009	0.000 ± 0.000	-	-	-	-	-
CLS2 + sparse fast (k-WTA 5%) [predict: max_conf]	3	0.6595 ± 0.0190	0.4206 ± 0.0230	0.578 ± 0.023	0.080 ± 0.005	3.27×10^{12}	269,322	0.857 ± 0.017	0.512 ± 0.004
CLS2 + sparse fast 10% + magnitude prune 5%/sleep [predict: fast]	3	0.8703 ± 0.0059	0.1528 ± 0.0075	0.846 ± 0.008	-	-	-	-	-
CLS2 + sparse fast 10% + magnitude prune 5%/sleep [predict: max_conf]	3	0.1969 ± 0.0002	0.9973 ± 0.0004	0.000 ± 0.000	-	-	-	-	-
CLS2 + sparse fast 10% + magnitude prune 5%/sleep	3	0.6728 ± 0.0133	0.4036 ± 0.0168	0.595 ± 0.017	0.079 ± 0.003	3.27×10^{12}	208,514	0.857 ± 0.013	0.510 ± 0.001
CLS2 + sparse fast 10%, sleep 300 steps lr 1e-3 [predict: fast]	3	0.1968 ± 0.0030	0.1548 ± 0.0039	0.844 ± 0.004	-	-	-	-	-
CLS2 + sparse fast 10%, sleep 300 steps lr 1e-3 [predict: max_conf]	3	0.1968 ± 0.0004	0.9957 ± 0.0021	0.000 ± 0.000	-	-	-	-	-
CLS2 + sleep decay; one-shot 10% at boundary [predict: fast]	3	0.8679 ± 0.0105	0.1510 ± 0.0145	0.848 ± 0.015	0.080 ± 0.007	2.25×10^{12}	269,322	0.857 ± 0.006	0.512 ± 0.006
CLS2 + sleep decay; one-shot 10% at boundary [predict: max_conf]	3	0.1972 ± 0.0002	0.9965 ± 0.0006	0.000 ± 0.000	-	-	-	-	-
CLS2 + sleep decay; periodic gradual, total 10% [predict: fast]	3	0.6579 ± 0.0100	0.4225 ± 0.0129	0.576 ± 0.013	0.077 ± 0.003	3.27×10^{12}	269,322	0.851 ± 0.010	0.508 ± 0.003
CLS2 + sleep decay; periodic gradual, total 10% [predict: max_conf]	3	0.1972 ± 0.0002	0.9965 ± 0.0006	0.000 ± 0.000	-	-	-	-	-
CLS2 + sleep decay; periodic gradual, total 10%	3	0.6313 ± 0.0025	0.4560 ± 0.0030	0.543 ± 0.003	0.072 ± 0.002	3.27×10^{12}	269,322	0.855 ± 0.013	0.507 ± 0.002
CLS2 + sleep decay; periodic gradual, total 3%	3	0.1972 ± 0.0002	0.9963 ± 0.0006	0.000 ± 0.000	-	-	-	-	-
CLS2 + sleep decay; periodic gradual, total 3%	3	0.6574 ± 0.0137	0.4233 ± 0.0165	0.576 ± 0.017	0.074 ± 0.008	3.27×10^{12}	269,322	0.860 ± 0.009	0.509 ± 0.001
CLS2 + sleep decay; continuous weight decay 1e-4 [predict: fast]	3	0.8759 ± 0.0087	0.1479 ± 0.0108	0.851 ± 0.011	-	-	-	-	-
CLS2 + sleep decay; continuous weight decay 1e-4 [predict: max_conf]	3	0.1972 ± 0.0002	0.9965 ± 0.0006	0.000 ± 0.000	-	-	-	-	-
CLS2 + sleep decay; continuous weight decay 1e-4	3	0.6510 ± 0.0071	0.4315 ± 0.0086	0.568 ± 0.009	0.078 ± 0.001	3.27×10^{12}	269,322	0.851 ± 0.012	0.507 ± 0.002
CLS2 (fast width 256), no distillation [predict: fast]	3	0.8710 ± 0.0022	0.1523 ± 0.0043	0.847 ± 0.004	-	-	-	-	-
CLS2 (fast width 256), no distillation [predict: max_conf]	3	0.1965 ± 0.0005	0.9972 ± 0.0004	0.000 ± 0.000	-	-	-	-	-
CLS2 (fast width 256), no distillation	3	0.6757 ± 0.0300	0.3995 ± 0.0374	0.599 ± 0.038	0.093 ± 0.002	1.31×10^{12}	269,322	0.864 ± 0.007	0.510 ± 0.003
CLS2 (fast width 256), global logit distillation [predict: fast]	3	0.8547 ± 0.0032	0.1718 ± 0.0041	0.827 ± 0.004	-	-	-	-	-
CLS2 (fast width 256), global logit distillation [predict: max_conf]	3	0.1966 ± 0.0005	0.9963 ± 0.0003	0.000 ± 0.000	-	-	-	-	-
CLS2 (fast 256) + global logit distillation	3	0.6007 ± 0.0226	0.4918 ± 0.0277	0.506 ± 0.028	0.145 ± 0.005	1.31×10^{12}	269,322	0.811 ± 0.009	0.519 ± 0.003
CLS2 (fast 256) + local per-layer feature distillation [predict: fast]	3	0.7928 ± 0.0044	0.2500 ± 0.0049	0.748 ± 0.005	-	-	-	-	-
CLS2 (fast 256) + local per-layer feature distillation [predict: max_conf]	3	0.1966 ± 0.0005	0.9963 ± 0.0003	0.000 ± 0.000	-	-	-	-	-
CLS2 (fast 256) + local per-layer feature distillation	3	0.6368 ± 0.0127	0.4468 ± 0.0151	0.552 ± 0.015	0.122 ± 0.012	1.31×10^{12}	269,322	0.874 ± 0.007	0.508 ± 0.002
CLS2 (fast 256) + local features + logits [predict: fast]	3	0.8131 ± 0.0085	0.2209 ± 0.0108	0.778 ± 0.011	-	-	-	-	-
CLS2 (fast 256) + local features + logits [predict: max_conf]	3	0.1966 ± 0.0005	0.9963 ± 0.0003	0.000 ± 0.000	-	-	-	-	-
CLS2 (fast 256) + local features + logits	3	0.5883 ± 0.0223	0.5050 ± 0.0276	0.492 ± 0.028	0.196 ± 0.013	1.31×10^{12}	269,322	0.813 ± 0.005	0.518 ± 0.002
CLS2 (fast 256) + local features + logits	3	0.7394 ± 0.0005	0.3062 ± 0.0066	0.691 ± 0.007	0.005 ± 0.002	8.79×10^{11}	269,322	0.934 ± 0.005	-
joint retraining (upper bound)	3	0.9762 ± 0.0009	0.0086 ± 0.0031	0.991 ± 0.003	-	-	-	-	-

Table 11: Split MNIST, class-incremental (single 10-way head, 5 tasks of 2 classes), 3 seeds. Rows marked [predict: ...] use an alternative read-out of the same run. Unknown-class AUROC: how well the confidence separates test samples of classes not yet learned (0.5 = chance).

method	seeds	avg acc (final)	forgetting	retention	ECE (all classes)	train FLOPs	active params (predictor)	unknown-class AUROC (softmax)	unknown-class AUROC (fast/slow agreement)
fine-tune (no protection)	3	0.4894 ± 0.0151	0.5384 ± 0.0166	0.447 ± 0.017	0.502 ± 0.043	2.90 × 10 ¹²	269,322	-	-
experience replay (buffer 500)	3	0.8739 ± 0.0103	0.1121 ± 0.0115	0.885 ± 0.012	0.187 ± 0.042	5.81 × 10 ¹²	269,322	-	-
ER, class-balanced batches (buffer 500)	3	0.8271 ± 0.0142	0.1646 ± 0.0157	0.831 ± 0.016	0.214 ± 0.017	1.16 × 10 ¹³	269,322	-	-
ER, class-balanced batches (buffer 500)	3	0.8828 ± 0.0037	0.0764 ± 0.0050	0.921 ± 0.005	0.140 ± 0.012	2.36 × 10 ¹²	269,322	-	-
CLS + sleep (replay + distill)	3	0.8780 ± 0.0038	0.0939 ± 0.0047	0.902 ± 0.005	0.110 ± 0.020	6.00 × 10 ¹²	269,322	-	-
CLS + sleep (no distill)	3	0.8856 ± 0.0006	0.0847 ± 0.0002	0.912 ± 0.000	0.125 ± 0.005	6.00 × 10 ¹²	269,322	-	-
CLS + sleep + downscale 0.1	3	0.8269 ± 0.0064	0.1512 ± 0.0070	0.843 ± 0.007	0.151 ± 0.022	6.00 × 10 ¹²	269,322	-	-
CLS + sleep + prune 5%/sleep (magnitude)	3	0.8731 ± 0.0040	0.0973 ± 0.0042	0.899 ± 0.004	0.130 ± 0.022	6.00 × 10 ¹²	161,462	-	-
CLS + sleep + prune 5%/sleep (activity)	3	0.8148 ± 0.0026	0.1389 ± 0.0029	0.833 ± 0.003	0.195 ± 0.029	6.00 × 10 ¹²	161,462	-	-
CLS2: balanced sleep replay; no distill, dense fast	3	0.8366 ± 0.0076	0.1382 ± 0.0088	0.850 ± 0.009	0.120 ± 0.029	6.00 × 10 ¹²	161,462	-	-
CLS2: balanced sleep replay; no distill, dense fast	3	0.4026 ± 0.0574	0.6295 ± 0.0630	0.351 ± 0.065	-	-	-	-	-
CLS2: balanced sleep replay; no distill, dense fast	3	0.8577 ± 0.0064	0.1256 ± 0.0070	0.871 ± 0.007	-	-	-	-	-
CLS2 + sparse fast (k-WTA 10%) [predict: max_conf]	3	0.8903 ± 0.0020	0.0662 ± 0.0024	0.931 ± 0.003	0.109 ± 0.022	1.18 × 10 ¹³	269,322	-	-
CLS2 + sparse fast (k-WTA 10%) [predict: fast]	3	0.4661 ± 0.0150	0.5601 ± 0.0169	0.423 ± 0.017	-	-	-	-	-
CLS2 + sparse fast (k-WTA 10%) [predict: max_conf]	3	0.8708 ± 0.0145	0.1119 ± 0.0161	0.885 ± 0.017	-	-	-	-	-
CLS2 + sparse fast (k-WTA 10%) [predict: fast]	3	0.8903 ± 0.0020	0.0662 ± 0.0024	0.931 ± 0.003	0.109 ± 0.022	1.18 × 10 ¹³	269,322	-	-
CLS2 + sparse fast 10%, sleep 300 steps lr 1e-3	3	0.4080 ± 0.0138	0.6248 ± 0.0162	0.357 ± 0.016	-	-	-	-	-
CLS2 + sparse fast 10%, sleep 300 steps lr 1e-3	3	0.8503 ± 0.0029	0.1330 ± 0.0034	0.863 ± 0.003	-	-	-	-	-
CLS2 + sparse fast 10%, sleep 300 steps lr 1e-3	3	0.8503 ± 0.0029	0.1330 ± 0.0034	0.863 ± 0.003	0.097 ± 0.022	9.97 × 10 ¹²	269,322	-	-
CLS2 + sleep decay: one-shot 10% at boundary [predict: fast]	3	0.8851 ± 0.0027	0.0617 ± 0.0043	0.935 ± 0.004	-	-	-	-	-
CLS2 + sleep decay: one-shot 10% at boundary [predict: max_conf]	3	0.4422 ± 0.0101	0.5866 ± 0.0112	0.396 ± 0.012	-	-	-	-	-
CLS2 + sleep decay: one-shot 10% at boundary [predict: fast]	3	0.8163 ± 0.0127	0.1725 ± 0.0142	0.823 ± 0.015	-	-	-	-	-
CLS2 + sleep decay: one-shot 10% at boundary [predict: max_conf]	3	0.8503 ± 0.0042	0.1125 ± 0.0042	0.882 ± 0.004	0.136 ± 0.004	1.18 × 10 ¹³	269,322	-	-
CLS2 + sleep decay: periodic gradual, total 10% [predict: fast]	3	0.4422 ± 0.0101	0.5866 ± 0.0112	0.396 ± 0.012	-	-	-	-	-
CLS2 + sleep decay: periodic gradual, total 10% [predict: max_conf]	3	0.8139 ± 0.0164	0.1753 ± 0.0184	0.820 ± 0.019	-	-	-	-	-
CLS2 + sleep decay: periodic gradual, total 10% [predict: fast]	3	0.8550 ± 0.0035	0.1075 ± 0.0028	0.888 ± 0.003	0.118 ± 0.008	1.18 × 10 ¹³	269,322	-	-
CLS2 + sleep decay: periodic gradual, total 10% [predict: max_conf]	3	0.4422 ± 0.0101	0.5866 ± 0.0112	0.396 ± 0.012	-	-	-	-	-
CLS2 + sleep decay: periodic gradual, total 3% [predict: fast]	3	0.8567 ± 0.0129	0.1277 ± 0.0146	0.869 ± 0.015	-	-	-	-	-
CLS2 + sleep decay: periodic gradual, total 3% [predict: max_conf]	3	0.8873 ± 0.0047	0.0718 ± 0.0047	0.925 ± 0.005	0.098 ± 0.011	1.18 × 10 ¹³	269,322	-	-
CLS2 + sleep decay: continuous weight decay 1e-4 [predict: fast]	3	0.4422 ± 0.0101	0.5866 ± 0.0112	0.396 ± 0.012	-	-	-	-	-
CLS2 + sleep decay: continuous weight decay 1e-4 [predict: max_conf]	3	0.8514 ± 0.0155	0.1338 ± 0.0173	0.862 ± 0.018	-	-	-	-	-
CLS2 + sleep decay: continuous weight decay 1e-4 [predict: fast]	3	0.8850 ± 0.0058	0.0743 ± 0.0056	0.922 ± 0.006	0.094 ± 0.009	1.18 × 10 ¹³	269,322	-	-
CLS2 (fast width 256), no distillation [predict: fast]	3	0.4215 ± 0.0056	0.6076 ± 0.0068	0.373 ± 0.007	-	-	-	-	-
CLS2 (fast width 256), no distillation [predict: max_conf]	3	0.8705 ± 0.0033	0.1110 ± 0.0035	0.886 ± 0.004	-	-	-	-	-
CLS2 (fast width 256), no distillation	3	0.8939 ± 0.0023	0.0636 ± 0.0019	0.934 ± 0.002	0.107 ± 0.010	4.75 × 10 ¹²	269,322	-	-
CLS2 (fast 256) + global logit distillation [predict: fast]	3	0.4300 ± 0.0207	0.5982 ± 0.0228	0.382 ± 0.024	-	-	-	-	-
CLS2 (fast 256) + global logit distillation [predict: max_conf]	3	0.8412 ± 0.0043	0.1426 ± 0.0041	0.853 ± 0.004	-	-	-	-	-
CLS2 (fast 256) + global logit distillation	3	0.8782 ± 0.0064	0.0941 ± 0.0070	0.902 ± 0.007	0.120 ± 0.011	4.75 × 10 ¹²	269,322	-	-
CLS2 (fast 256) + local per-layer feature distillation [predict: fast]	3	0.4300 ± 0.0207	0.5982 ± 0.0228	0.382 ± 0.024	-	-	-	-	-
CLS2 (fast 256) + local per-layer feature distillation [predict: max_conf]	3	0.8258 ± 0.0067	0.1580 ± 0.0075	0.837 ± 0.008	-	-	-	-	-
CLS2 (fast 256) + local per-layer feature distillation	3	0.8516 ± 0.0047	0.1067 ± 0.0049	0.888 ± 0.005	0.178 ± 0.021	4.75 × 10 ¹²	269,322	-	-
CLS2 (fast 256) + local features + logits [predict: fast]	3	0.4300 ± 0.0207	0.5982 ± 0.0228	0.382 ± 0.024	-	-	-	-	-
CLS2 (fast 256) + local features + logits [predict: fast]	3	0.8087 ± 0.0088	0.1744 ± 0.0095	0.820 ± 0.010	-	-	-	-	-
CLS2 (fast 256) + local features + logits [predict: max_conf]	3	0.8355 ± 0.0079	0.1266 ± 0.0076	0.867 ± 0.008	0.172 ± 0.017	4.75 × 10 ¹²	269,322	-	-
CLS2 (fast 256) + local features + logits	3	0.8355 ± 0.0079	0.1266 ± 0.0076	0.867 ± 0.008	0.007 ± 0.002	8.87 × 10 ¹²	269,322	-	-
joint retraining (upper bound)	3	0.9633 ± 0.0061	0.0034 ± 0.0005	0.998 ± 0.001	-	-	-	-	-

Table 12: Permuted MNIST, domain-incremental (10 tasks), 3 seeds. Columns as in Table 11.

score	substitute	skip frac	test acc	ECE	FLOPs ratio
baseline	-	0	0.9827 ± 0.0003	0.005 ± 0.001	1.000
predicted	identity	0.1	0.9745 ± 0.0026	0.007 ± 0.001	0.935
predicted	identity	0.2	0.9623 ± 0.0073	0.010 ± 0.003	0.851
predicted	identity	0.3	0.9494 ± 0.0112	0.013 ± 0.005	0.767
predicted	identity	0.4	0.9395 ± 0.0149	0.015 ± 0.006	0.683
predicted	identity	0.5	0.9290 ± 0.0203	0.019 ± 0.008	0.616
predicted	identity	0.6	0.9157 ± 0.0247	0.024 ± 0.010	0.531
predicted	identity	0.7	0.9066 ± 0.0252	0.027 ± 0.010	0.447
predicted	identity	0.8	0.8927 ± 0.0177	0.032 ± 0.007	0.363
predicted	identity_late_only	0.3	0.9807 ± 0.0005	0.005 ± 0.000	0.884
predicted	identity_late_only	0.5	0.9800 ± 0.0011	0.006 ± 0.000	0.808
predicted	identity_late_only	0.7	0.9798 ± 0.0011	0.006 ± 0.001	0.724
predicted	predicted	0.1	0.9789 ± 0.0014	0.006 ± 0.001	0.935
predicted	predicted	0.2	0.9730 ± 0.0031	0.006 ± 0.000	0.851
predicted	predicted	0.3	0.9678 ± 0.0057	0.007 ± 0.001	0.767
predicted	predicted	0.4	0.9645 ± 0.0079	0.008 ± 0.002	0.683
predicted	predicted	0.5	0.9601 ± 0.0088	0.010 ± 0.003	0.616
predicted	predicted	0.6	0.9557 ± 0.0094	0.012 ± 0.003	0.531
predicted	predicted	0.7	0.9502 ± 0.0079	0.014 ± 0.003	0.447
predicted	predicted	0.8	0.9439 ± 0.0063	0.016 ± 0.003	0.363
random	identity	0.1	0.9796 ± 0.0006	0.005 ± 0.000	0.935
random	identity	0.2	0.9747 ± 0.0013	0.007 ± 0.002	0.851
random	identity	0.3	0.9671 ± 0.0016	0.008 ± 0.001	0.767
random	identity	0.4	0.9574 ± 0.0030	0.011 ± 0.001	0.683
random	identity	0.5	0.9448 ± 0.0016	0.013 ± 0.003	0.616
random	identity	0.6	0.9266 ± 0.0065	0.020 ± 0.003	0.531
random	identity	0.7	0.9028 ± 0.0101	0.030 ± 0.004	0.447
random	identity	0.8	0.8785 ± 0.0215	0.037 ± 0.012	0.363
oracle	identity	0.1	0.9774 ± 0.0011	0.006 ± 0.001	0.935
oracle	identity	0.2	0.9679 ± 0.0018	0.008 ± 0.001	0.851
oracle	identity	0.3	0.9557 ± 0.0027	0.011 ± 0.003	0.767
oracle	identity	0.4	0.9446 ± 0.0050	0.015 ± 0.003	0.683
oracle	identity	0.5	0.9353 ± 0.0039	0.018 ± 0.003	0.616
oracle	identity	0.6	0.9242 ± 0.0056	0.021 ± 0.002	0.531
oracle	identity	0.7	0.9121 ± 0.0081	0.025 ± 0.004	0.447
oracle	identity	0.8	0.8967 ± 0.0126	0.030 ± 0.008	0.363

Table 13: MNIST ViT: post-hoc token skipping without fine-tuning, 3 seeds. Score: which tokens are skipped (predicted change, random, or the oracle true residual norm); substitute: what replaces a skipped token (identity, identity in the late blocks only, or the predicted residual).

score	substitute	skip frac	test acc	ECE	FLOPs ratio
baseline	-	0	0.8186	0.047	1.000
predicted	identity	0.1	0.7851	0.054	0.936
predicted	identity	0.2	0.7269	0.058	0.845
predicted	identity	0.3	0.6647	0.095	0.768
predicted	identity	0.4	0.5876	0.143	0.677
predicted	identity	0.5	0.5149	0.194	0.600
predicted	identity	0.6	0.4349	0.252	0.522
predicted	identity	0.7	0.3510	0.317	0.432
predicted	identity	0.8	0.3003	0.343	0.354
predicted	identity__late__only	0.3	0.8012	0.050	0.884
predicted	identity__late__only	0.5	0.7854	0.056	0.800
predicted	identity__late__only	0.7	0.7685	0.055	0.716
predicted	predicted	0.1	0.8020	0.052	0.936
predicted	predicted	0.2	0.7685	0.046	0.845
predicted	predicted	0.3	0.7300	0.053	0.768
predicted	predicted	0.4	0.6843	0.069	0.677
predicted	predicted	0.5	0.6373	0.093	0.600
predicted	predicted	0.6	0.5735	0.135	0.522
predicted	predicted	0.7	0.5079	0.173	0.432
predicted	predicted	0.8	0.4422	0.218	0.354
random	identity	0.1	0.7982	0.051	0.936
random	identity	0.2	0.7663	0.051	0.845
random	identity	0.3	0.7245	0.053	0.768
random	identity	0.4	0.6544	0.093	0.677
random	identity	0.5	0.5854	0.140	0.600
random	identity	0.6	0.5094	0.194	0.522
random	identity	0.7	0.4228	0.250	0.432
random	identity	0.8	0.3529	0.299	0.354
oracle	identity	0.1	0.7941	0.054	0.936
oracle	identity	0.2	0.7462	0.060	0.845
oracle	identity	0.3	0.6953	0.077	0.768
oracle	identity	0.4	0.6269	0.121	0.677
oracle	identity	0.5	0.5531	0.176	0.600
oracle	identity	0.6	0.4667	0.237	0.522
oracle	identity	0.7	0.3783	0.306	0.432
oracle	identity	0.8	0.3347	0.312	0.354

Table 14: CIFAR-10 ViT: post-hoc token skipping without fine-tuning, one seed. Columns as in Table 13.

train mode / late attn	seeds	no skip	warm-up only, skip $\epsilon_{\max}$	skip 0	skip 0.3	skip 0.5	skip 0.7	skip 0.8	skip 0.9	FLOPs ratio (by skip level)	late blocks removed: acc, no fine-tune / fine-tuned
late blocks removed, same fine-tuning (control)	3	0.9827 $\pm$ 0.0003	—	—	—	—	—	—	—	0.50	—
layerwise / late attn full	3	0.9827 $\pm$ 0.0003	0.9806 $\pm$ 0.0004	0.9785 $\pm$ 0.0012	0.9787 $\pm$ 0.0016	0.9788 $\pm$ 0.0015	0.9788 $\pm$ 0.0016	0.9787 $\pm$ 0.0016	0.9787 $\pm$ 0.0015	1.01 / 0.89 / 0.81 / 0.73 / 0.69 / 0.65	0.8911 $\pm$ 0.0126 / 0.9657 $\pm$ 0.0028
(eval thalamic)	3	—	—	—	0.9788 $\pm$ 0.0015	0.9789 $\pm$ 0.0016	0.9786 $\pm$ 0.0015	0.9786 $\pm$ 0.0013	0.9784 $\pm$ 0.0013	—	—
(eval random)	3	—	—	—	0.9783 $\pm$ 0.0015	0.9787 $\pm$ 0.0009	0.9786 $\pm$ 0.0014	0.9784 $\pm$ 0.0014	0.9784 $\pm$ 0.0014	—	—
random / late attn full	3	0.9827 $\pm$ 0.0003	0.9809 $\pm$ 0.0003	0.9794 $\pm$ 0.0020	0.9791 $\pm$ 0.0019	0.9792 $\pm$ 0.0015	0.9790 $\pm$ 0.0016	0.9787 $\pm$ 0.0016	0.9787 $\pm$ 0.0016	1.01 / 0.89 / 0.81 / 0.73 / 0.69 / 0.65	—
(eval thalamic)	3	—	—	—	0.9795 $\pm$ 0.0019	0.9793 $\pm$ 0.0021	0.9793 $\pm$ 0.0021	0.9791 $\pm$ 0.0021	0.9790 $\pm$ 0.0017	—	—
(eval layerwise)	3	—	—	—	0.9789 $\pm$ 0.0016	0.9789 $\pm$ 0.0018	0.9790 $\pm$ 0.0018	0.9789 $\pm$ 0.0016	0.9789 $\pm$ 0.0017	—	—
thalamic / late attn full / identity substitution	3	0.9827 $\pm$ 0.0003	0.9805 $\pm$ 0.0015	0.9785 $\pm$ 0.0015	0.9788 $\pm$ 0.0017	0.9789 $\pm$ 0.0017	0.9789 $\pm$ 0.0018	0.9787 $\pm$ 0.0015	0.9787 $\pm$ 0.0017	1.01 / 0.89 / 0.81 / 0.73 / 0.69 / 0.65	—
(eval layerwise)	3	—	—	—	0.9787 $\pm$ 0.0017	0.9788 $\pm$ 0.0018	0.9788 $\pm$ 0.0014	0.9788 $\pm$ 0.0015	0.9788 $\pm$ 0.0016	—	—
(eval random)	3	—	—	—	0.9780 $\pm$ 0.0016	0.9790 $\pm$ 0.0020	0.9785 $\pm$ 0.0015	0.9790 $\pm$ 0.0016	0.9790 $\pm$ 0.0018	—	—
thalamic / late attn full	3	0.9827 $\pm$ 0.0003	0.9810 $\pm$ 0.0010	0.9788 $\pm$ 0.0015	0.9790 $\pm$ 0.0019	0.9787 $\pm$ 0.0018	0.9786 $\pm$ 0.0016	0.9783 $\pm$ 0.0014	0.9783 $\pm$ 0.0012	1.01 / 0.89 / 0.81 / 0.73 / 0.69 / 0.65	—
(eval layerwise)	3	—	—	—	0.9783 $\pm$ 0.0019	0.9782 $\pm$ 0.0021	0.9781 $\pm$ 0.0019	0.9780 $\pm$ 0.0018	0.9779 $\pm$ 0.0016	—	—
(eval random)	3	—	—	—	0.9784 $\pm$ 0.0018	0.9788 $\pm$ 0.0013	0.9786 $\pm$ 0.0015	0.9780 $\pm$ 0.0013	0.9781 $\pm$ 0.0012	—	—
thalamic / late attn pre	3	0.9168 $\pm$ 0.0088	0.9140 $\pm$ 0.0089	0.9773 $\pm$ 0.0021	0.9774 $\pm$ 0.0027	0.9773 $\pm$ 0.0029	0.9773 $\pm$ 0.0027	0.9771 $\pm$ 0.0026	0.9773 $\pm$ 0.0022	0.99 / 0.87 / 0.80 / 0.72 / 0.68 / 0.64	—
(eval layerwise)	3	—	—	—	0.9776 $\pm$ 0.0026	0.9773 $\pm$ 0.0025	0.9772 $\pm$ 0.0024	0.9771 $\pm$ 0.0023	0.9769 $\pm$ 0.0024	—	—
(eval random)	3	—	—	—	0.9771 $\pm$ 0.0019	0.9772 $\pm$ 0.0025	0.9772 $\pm$ 0.0020	0.9771 $\pm$ 0.0022	0.9772 $\pm$ 0.0019	—	—

Table 15: MNIST ViT: token skipping in the late blocks. Skipped tokens receive the predicted residual, except in the rows marked identity substitution, where they are left unchanged. Each cell is test accuracy at the given skip ratio after fine-tuning that ramps the skip ratio from 0 to 0.7; the warm-up-only column is measured after the one-epoch warm-up of predictor and router with the ViT frozen, at the maximum skip ratio and before fine-tuning. Rows marked (eval thalamic) evaluate the same model with the thalamic router choosing the skipped tokens. For the pre-routed variant (late attn pre) the no-skip and warm-up columns are measured right after the late-block attention is swapped for pre-routed attention and before fine-tuning, which is why they are low; the skip columns are after fine-tuning.

train mode / late attn	seeds	no skip	warm-up only, skip	$s_{\max}$	skip 0	skip 0.3	skip 0.5	skip 0.7	skip 0.8	skip 0.9	FLOPs ratio (by skip level)	late blocks removed: acc, no fine-tune / fine-tuned
late blocks removed, same fine-tuning (control)	1	0.8184									0.50	0.5106 / 0.7285
layerwise / late attn full	1	0.8184		0.7795	0.8011	0.8040	0.8028	0.8041	0.8008	0.7979	1.00 / 0.88 / 0.80 / 0.72 / 0.68 / 0.64	-
(eval thalamic)	1	-	-	-	-	0.8058	0.8065	0.8049	0.8043	0.8006	-	-
(eval random)	1	-	-	-	-	0.8059	0.8067	0.8069	0.8042	0.7986	-	-
random / late attn full	1	0.8184		0.7938	0.8067	0.8117	0.8091	0.8072	0.8007	0.7977	1.00 / 0.88 / 0.80 / 0.72 / 0.68 / 0.64	-
(eval thalamic)	1	-	-	-	-	0.8069	0.8080	0.8035	0.8011	0.7952	-	-
(eval layerwise)	1	-	-	-	-	0.8071	0.8026	0.8004	0.7965	0.7935	-	-
thalamic / late attn full / identity substitution	1	0.8184		0.7764	0.7913	0.8032	0.8082	0.8079	0.8035	0.7987	1.00 / 0.88 / 0.80 / 0.72 / 0.68 / 0.64	-
(eval layerwise)	1	-	-	-	-	0.7999	0.8003	0.8004	0.7963	0.7905	-	-
(eval random)	1	-	-	-	-	0.8041	0.8072	0.8071	0.8031	0.7953	-	-
thalamic / late attn full	1	0.8184		0.7860	0.8018	0.8098	0.8110	0.8079	0.8053	0.8015	1.00 / 0.88 / 0.80 / 0.72 / 0.68 / 0.64	-
(eval layerwise)	1	-	-	-	-	0.8090	0.8065	0.8039	0.8031	0.7976	-	-
(eval random)	1	-	-	-	-	0.8091	0.8110	0.8096	0.8040	0.7989	-	-
thalamic / late attn pre	1	0.5679		0.5685	0.8006	0.8039	0.8028	0.8015	0.8026	0.7942	0.99 / 0.87 / 0.79 / 0.71 / 0.68 / 0.63	-
(eval layerwise)	1	-	-	-	-	0.8029	0.8020	0.7973	0.7961	0.7935	-	-
(eval random)	1	-	-	-	-	0.8053	0.8024	0.8044	0.8003	0.7939	-	-

Table 16: CIFAR-10 ViT: token skipping in the late blocks, as in Table 15. late attn pre replaces the late-block attention by pre-routed attention (25% of keys).

mode	keep ratio	k (keys/query)	seeds	test acc	ECE	attn FLOPs/block (dense)	attn FLOPs/block (sparsity exploited)	total FLOPs ratio
full	1	50	3	0.9794 ± 0.0011	0.005 ± 0.001	1.28×10^6	1.28×10^6	1.000
post	0.5	25	3	0.9798 ± 0.0010	0.004 ± 0.000	1.28×10^6	9.60×10^5	0.985
post	0.25	13	3	0.9808 ± 0.0011	0.005 ± 0.001	1.28×10^6	8.06×10^5	0.977
post	0.1	5	3	0.9814 ± 0.0001	0.004 ± 0.000	1.28×10^6	7.04×10^5	0.973
pre	0.5	25	3	0.9778 ± 0.0012	0.006 ± 0.000	8.00×10^5	8.00×10^5	0.977
pre	0.25	13	3	0.9804 ± 0.0007	0.004 ± 0.001	4.93×10^5	4.93×10^5	0.962
pre	0.1	5	3	0.9808 ± 0.0002	0.005 ± 0.001	2.88×10^5	2.88×10^5	0.953

Table 17: MNIST ViT (50 tokens): pre-routed sparse attention (an 8-dimensional router selects k keys per query before attention) against post-hoc top- k masking and full attention, 3 seeds. Attention FLOPs per block are given for the dense computation and with the sparsity exploited (post-hoc masking still computes the full score matrix and saves only the value aggregation; pre-routing computes scores for k keys only); the total ratio replaces the dense attention FLOPs of every block by the mode’s attention FLOPs and divides by the full model’s FLOPs.

mode	keep ratio	k (keys/query)	seeds	test acc	ECE	attn FLOPs/block (dense)	attn FLOPs/block (sparsity exploited)	total FLOPs ratio
full	1	65	1	0.7940	0.035	3.24×10^6	3.24×10^6	1.000
post	0.25	17	1	0.8027	0.033	3.24×10^6	2.05×10^6	0.980
pre	0.25	17	1	0.8003	0.036	1.05×10^6	1.05×10^6	0.964

Table 18: CIFAR-10 ViT (65 tokens): pre-routed sparse attention, one seed. Columns as in Table 17.

group	method	seeds	avg acc (final)	delta vs ref (pp)	forgetting	retention	ECE	train FLOPs
sleep decay variants	CLS2 + sparse fast (k-WTA 10%) (reference)	3	0.8703 ± 0.0059	-	0.1528 ± 0.0075	0.846 ± 0.008	0.080 ± 0.005	3.27×10^{12}
sleep decay variants	CLS2 + sleep decay: one-shot 10% at boundary	3	0.8735 ± 0.0015	+0.32	0.1481 ± 0.0024	0.851 ± 0.002	0.077 ± 0.003	3.27×10^{12}
sleep decay variants	CLS2 + sleep decay: periodic gradual, total 10%	3	0.8729 ± 0.0020	+0.27	0.1498 ± 0.0030	0.850 ± 0.003	0.072 ± 0.002	3.27×10^{12}
sleep decay variants	CLS2 + sleep decay: periodic gradual, total 3%	3	0.8750 ± 0.0087	+0.47	0.1479 ± 0.0108	0.851 ± 0.011	0.074 ± 0.008	3.27×10^{12}
sleep decay variants	CLS2 + sleep decay: continuous weight decay 1e-4	3	0.8710 ± 0.0022	+0.08	0.1523 ± 0.0043	0.847 ± 0.004	0.078 ± 0.001	3.27×10^{12}
distillation locality (fast width 256)	CLS2 (fast width 256), no distillation (reference)	3	0.8547 ± 0.0032	-	0.1718 ± 0.0041	0.827 ± 0.004	0.093 ± 0.002	1.31×10^{12}
distillation locality (fast width 256)	CLS2 (fast 256) + global logit distillation	3	0.7928 ± 0.0044	-6.19	0.2509 ± 0.0049	0.748 ± 0.005	0.145 ± 0.005	1.31×10^{12}
distillation locality (fast width 256)	CLS2 (fast 256) + local per-layer feature distillation	3	0.8131 ± 0.0085	-4.16	0.2209 ± 0.0108	0.778 ± 0.011	0.122 ± 0.012	1.31×10^{12}
distillation locality (fast width 256)	CLS2 (fast 256) + local features + logits	3	0.7394 ± 0.0025	-11.53	0.3062 ± 0.0066	0.691 ± 0.007	0.196 ± 0.013	1.31×10^{12}

Table 19: Split MNIST: sleep-decay variants and distillation locality on the sparse redesign (CLS2, k -WTA 10% fast network), 3 seeds. Delta is the change in final average accuracy against the reference row of each group, in points.

group	method	seeds	avg acc (final)	delta vs ref (pp)	forgetting	retention	ECE	train FLOPs
sleep decay variants	CLS2 + sparse fast (k-WTA 10%) (reference)	3	0.8903 ± 0.0020	-	0.0662 ± 0.0024	0.931 ± 0.003	0.109 ± 0.022	1.18×10^{13}
sleep decay variants	CLS2 + sleep decay: one-shot 10% at boundary	3	0.8503 ± 0.0042	-4.00	0.1125 ± 0.0042	0.882 ± 0.004	0.136 ± 0.004	1.18×10^{13}
sleep decay variants	CLS2 + sleep decay: periodic gradual, total 10%	3	0.8550 ± 0.0035	-3.53	0.1075 ± 0.0028	0.888 ± 0.003	0.118 ± 0.008	1.18×10^{13}
sleep decay variants	CLS2 + sleep decay: periodic gradual, total 3%	3	0.8873 ± 0.0047	-0.30	0.0718 ± 0.0047	0.925 ± 0.005	0.098 ± 0.011	1.18×10^{13}
sleep decay variants	CLS2 + sleep decay: continuous weight decay 1e-4	3	0.8850 ± 0.0058	-0.53	0.0743 ± 0.0056	0.922 ± 0.006	0.094 ± 0.009	1.18×10^{13}
distillation locality (fast width 256)	CLS2 (fast width 256), no distillation (reference)	3	0.8939 ± 0.0023	-	0.0636 ± 0.0019	0.934 ± 0.002	0.107 ± 0.010	4.75×10^{12}
distillation locality (fast width 256)	CLS2 (fast 256) + global logit distillation	3	0.8782 ± 0.0064	-1.58	0.0941 ± 0.0070	0.902 ± 0.007	0.120 ± 0.011	4.75×10^{12}
distillation locality (fast width 256)	CLS2 (fast 256) + local per-layer feature distillation	3	0.8516 ± 0.0047	-4.23	0.1067 ± 0.0049	0.888 ± 0.005	0.178 ± 0.021	4.75×10^{12}
distillation locality (fast width 256)	CLS2 (fast 256) + local features + logits	3	0.8355 ± 0.0079	-5.85	0.1266 ± 0.0076	0.867 ± 0.008	0.172 ± 0.017	4.75×10^{12}

Table 20: Permuted MNIST: sleep-decay variants and distillation locality, 3 seeds. Columns as in Table 19.

density	arm	seeds	active/sample	test acc	ECE	infer FLOPs	cum. train FLOPs	Jaccard same	Jaccard diff	coverage
0.2	static: dense small (additive)	2	2,239,484	0.9093 $\pm$ 0.0000	0.035 $\pm$ 0.002	2.25 $\times 10^8$	6.74 $\times 10^{14}$	-	-	-
0.2	static: prune-during, magnitude (global)	2	2,232,870	0.9189 $\pm$ 0.0013	0.038 $\pm$ 0.000	3.45 $\times 10^8$	1.65 $\times 10^{15}$	-	-	-
0.2	dynamic, channel-level k-WTA input gating	2	2,233,536	0.7339 $\pm$ 0.0072	0.021 $\pm$ 0.000	2.27 $\times 10^8$	1.36 $\times 10^{15}$	0.192 $\pm$ 0.010	0.127 $\pm$ 0.002	1.000 $\pm$ 0.000
0.2	dynamic, connection-level, random selection	2	2,238,912	0.7614 $\pm$ 0.0023	0.024 $\pm$ 0.003	2.25 $\times 10^8$	1.36 $\times 10^{15}$	0.111 $\pm$ 0.000	0.111 $\pm$ 0.000	1.000 $\pm$ 0.000
0.2	dynamic, connection-level, local rule (abs(w) $\times$ input magnitude)	2	2,238,912	0.9183 $\pm$ 0.0010	0.038 $\pm$ 0.000	2.25 $\times 10^8$	1.36 $\times 10^{15}$	0.613 $\pm$ 0.005	0.462 $\pm$ 0.008	0.429 $\pm$ 0.003
0.05	static: dense small (additive)	3	562,229	0.8866 $\pm$ 0.0018	0.030 $\pm$ 0.002	5.58 $\times 10^7$	1.67 $\times 10^{14}$	-	-	-
0.05	static: prune-after, one-shot magnitude + fine-tune	3	558,218	0.9227 $\pm$ 0.0006	0.035 $\pm$ 0.001	2.71 $\times 10^8$	3.74 $\times 10^{15}$	-	-	-
0.05	static: RigL	3	558,213	0.9097 $\pm$ 0.0016	0.034 $\pm$ 0.001	1.39 $\times 10^8$	4.17 $\times 10^{14}$	-	-	-
0.05	static: prune-during, magnitude (ERK)	3	558,213	0.9203 $\pm$ 0.0009	0.034 $\pm$ 0.001	1.39 $\times 10^8$	1.43 $\times 10^{15}$	-	-	-
0.05	static: prune-during, magnitude (global)	3	558,218	0.9196 $\pm$ 0.0023	0.034 $\pm$ 0.001	1.85 $\times 10^8$	1.33 $\times 10^{15}$	-	-	-
0.05	dynamic, channel-level k-WTA input gating	2	569,536	0.4486 $\pm$ 0.0036	0.016 $\pm$ 0.003	5.74 $\times 10^7$	9.83 $\times 10^{14}$	0.083 $\pm$ 0.002	0.052 $\pm$ 0.001	0.916 $\pm$ 0.008
0.05	dynamic, connection-level, random selection	2	564,160	0.5229 $\pm$ 0.0005	0.029 $\pm$ 0.001	5.91 $\times 10^7$	9.86 $\times 10^{14}$	0.026 $\pm$ 0.000	0.026 $\pm$ 0.000	1.000 $\pm$ 0.000
0.05	dynamic, connection-level, local rule (abs(w) $\times$ input magnitude)	2	564,160	0.8983 $\pm$ 0.0011	0.039 $\pm$ 0.003	5.91 $\times 10^7$	9.86 $\times 10^{14}$	0.439 $\pm$ 0.003	0.274 $\pm$ 0.002	0.207 $\pm$ 0.003
0.005	static: dense small (additive)	3	55,872	0.8131 $\pm$ 0.0026	0.021 $\pm$ 0.001	6.12 $\times 10^6$	1.84 $\times 10^{13}$	-	-	-
0.005	static: prune-after, one-shot magnitude + fine-tune	3	55,822	0.8364 $\pm$ 0.0015	0.016 $\pm$ 0.001	5.27 $\times 10^7$	3.41 $\times 10^{15}$	-	-	-
0.005	static: RigL	3	55,820	0.8486 $\pm$ 0.0030	0.026 $\pm$ 0.001	1.38 $\times 10^7$	4.15 $\times 10^{13}$	-	-	-
0.005	static: prune-during, magnitude (ERK)	3	55,820	0.8783 $\pm$ 0.0022	0.024 $\pm$ 0.001	1.38 $\times 10^7$	1.18 $\times 10^{15}$	-	-	-
0.005	static: prune-during, magnitude (global)	3	55,822	0.8896 $\pm$ 0.0003	0.027 $\pm$ 0.001	3.40 $\times 10^7$	1.09 $\times 10^{15}$	-	-	-
0.005	dynamic, channel-level k-WTA input gating	2	69,952	0.3404 $\pm$ 0.0296	0.021 $\pm$ 0.001	1.34 $\times 10^7$	8.80 $\times 10^{14}$	0.058 $\pm$ 0.007	0.036 $\pm$ 0.002	0.285 $\pm$ 0.021
0.005	dynamic, connection-level, random selection	2	63,424	0.1824 $\pm$ 0.0026	0.040 $\pm$ 0.002	9.32 $\times 10^6$	8.75 $\times 10^{14}$	0.002 $\pm$ 0.000	0.003 $\pm$ 0.000	0.633 $\pm$ 0.000
0.005	dynamic, connection-level, local rule (abs(w) $\times$ input magnitude)	2	63,424	0.7768 $\pm$ 0.0015	0.019 $\pm$ 0.001	9.32 $\times 10^6$	8.75 $\times 10^{14}$	0.333 $\pm$ 0.009	0.246 $\pm$ 0.006	0.033 $\pm$ 0.001

Table 21: Input-dependent (per-sample) pruning of a CIFAR-10 ResNet-18 against fixed masks at the same per-sample budget. Active weights are the per-sample expectation; Jaccard values are overlaps of the last-stage masks of two test images of the same or of different classes (200 images; a random mask gives 0.026); union coverage is the fraction of that stage’s connections used by at least one of the 200 images.

n_e	seeds	STDP acc	MLP acc	SNN SOP/img	MLP FLOPs/img	SNN FLOPs/img	SNN train ops	MLP train FLOPs	SNN to 80%	MLP to 80%	$w < 1\% w_{\max}$	time (s)
100	1	0.3697	0.9474	1.08×10^5	1.59×10^5	1.15×10^{10}	1.15×10^{10}	2.86×10^{10}	-	4992	0.549	550
400	3	0.8108 ± 0.0028	0.9649 ± 0.0021	4.33×10^5	6.35×10^5	4.60×10^{10}	4.60×10^{10}	1.14×10^{11}	38189 ± 492	4992 ± 0	0.577 ± 0.000	561 ± 9
1600	1	0.7289	0.9682	1.74×10^6	2.54×10^6	1.83×10^{11}	1.83×10^{11}	4.57×10^{11}	-	4992	0.287	576
control: MNIST class-mean background pixels	-	-	-	-	-	-	-	-	-	-	-	-
									-	0.498 (per class 0.42-0.70; zero in 99% of images 0.371)		

Table 22: Unsupervised STDP (Diehl & Cook) against a backprop MLP of the same width trained on the same samples. SOP = synaptic operations (event-driven, accumulated). No energy comparison is made: everything runs on a GPU.